1. This stuff is hard

You might think this is a strange way to start a book, but: concurrency is a difficult topic, and you might sometimes find yourself feeling a bit lost or a bit confused while you're learning about it.

I'm not saying that to put you off, or to make you feel somehow scared by all the topics we're going to cover. In fact, my goal here is quite the opposite: I want you to know that *everyone* finds this stuff hard. If you ever feel like you're struggling, it's okay – it's not you being stupid, it's just plain difficult stuff.

It's my goal to try to make this all as straightforward, understandable, and most importantly *applicable* as I can, so that anyone who is an intermediate to advanced Swift developer can apply these concepts to their projects immediately. If you're a beginner developer you're welcome to try following along, but honestly I would suggest you come back later – lots of theory isn't likely to stay in your head unless you have the chance to actually apply it.

Our brains are inherently not built to work concurrently, and as a result it's often hard to think about. So, one last time: this stuff is hard, and if you're finding it hard it's just a sign your brain is working correctly.

2. How to follow this guide

This guide is called Swift Concurrency by Example because it focuses on providing as many examples as possible. My goal is to stay laser-focused on real-world problems and real-world solutions, and to give clear, pragmatic advice on how to use specific techniques and approaches.

A lot of the time I've tried to make the chapters in this book answer specific questions such as "what are…" or "how to…", so that you can see exactly what problem is being solved and get straight there. That also means I've tried to get to the point as fast as possible and stay there so that you can get answers and build understanding quickly.

You can read this in a linear order if you want, or just dive in to a particular chapter that interests you – either one works.

Whatever you choose, you should know that his book was written using Xcode 16 with Swift 6 language mode enabled. Although most of it ought to work fine in Swift 5 language mode, it's not something I've specifically tested, and honestly enabling Swift 6 language mode will give you the smoothest journey in learning Swift concurrency.

3. Concurrency vs parallelism

When working through concurrency in Swift, there are two words we use a lot: *concurrency* and *parallelism*. We give them very specific meanings that might not quite match how you use them in English, so it's worth clarifying up front what they mean in this specific context.

Imagine a single-core CPU running a desktop operating system: the user can run lots of programs, use the internet, maybe play some music, and from the user's perspective all those things are happening at the same time. However, we know that isn't possible: they have a single-core CPU, which means it can literally only do *one* thing at a time.

What you're seeing here is called *concurrency*: our user's apps have been written in such a way that one CPU core can juggle them so they appear to be running at the same time. One starts, makes a tiny bit of progress on its work, then pauses so that another one can start, make a bit of progress on *its* work, then pause, and so on. Because the CPU flips between programs so quickly, they appear to be running all at the same time, when really they aren't.

As soon as you add a *second* CPU core to a computer, then things can run in *parallel*. This is when two or more programs run at the same instant: one on the first core, and another on the second. As you might imagine, this means work can happen up to twice as fast because two programs are able to run at the same time.

The really interesting stuff lies in our ability to split up work in a single program. Yes, having two CPU cores allows a computer to run two separate programs at the same time, but we can also split up our work into smaller parts called threads, and *those* can be run in parallel too.

This splitting up of functionality requires us to do work. You see, Swift can't decide by itself to run some parts of our code in parallel with other parts, because that would introduce a lot of surprising bugs. Instead, we have to tell Swift ahead of time which parts of our code can be split up if needed, and also tell it what we should do when those tasks complete.

When you boil it right down, that's the topic of this whole book: teaching Swift how it can split up the work in our programs so it runs as efficiently as possible. And it *is* about efficiency, because some Apple devices have many CPU cores – if your app is running full screen on that device and you're only ever using one of those cores, you're only getting a tiny fraction of the device's possible performance.

More importantly, you're also helping to make sure your app remains responsive the entire time. Imagine if your user interface froze up every time you were waiting for the response to a network request – it would be a pretty horrible experience, right?

There's a famous computer scientist called Rob Pike, and I think he explained the difference between concurrency and parallelism beautifully. Here's what he said:

> *"Concurrency is about dealing with many things at once, parallelism is about doing many things at once. Concurrency is a way to structure things so you can maybe use parallelism to do a better job."*

4. Understanding threads and queues

Every program launches with at least one thread where its work takes place, called the *main thread*. Super simple command-line apps for macOS might only ever have that one thread, iOS apps will have many more to do all sorts of other jobs, but either way that initial thread – the one the app is first launched with – always exists for the lifetime of the app, and it's always called the main thread.

This is important, because all your user interface work must take place on that main thread. Not some work some of the time, but *all* work *all* the time – if you try to update your UI from any other thread in your program you might find nothing happens, you might find your app crashes, or pretty much anywhere in between.

This rule exists for all apps that run on iOS, macOS, tvOS, and watchOS, and even though it's simple you will – *will* – forget it at some point in the future. It's like an initiation rite, except it happens more often than I'd like to admit even after years of programming.

Although Swift lets us create threads whenever we want, this is uncommon because it creates a lot of complexity. Each thread you create needs to run *somewhere*, and if you accidentally end up creating 40 threads when you have only 4 CPU cores, the system will need to spend a lot of time just swapping them.

Swapping threads is known as a *context switch*, and it has a performance cost: the system must stash away all the data the thread was using and remember how far it had progressed in its work, before giving another thread the chance to run. When this happens a lot – when you create many more threads compared to the number of available CPU cores – the cost of context switching grows high, and so it has a suitably disastrous-sounding name: *thread explosion*.

And so, apart from that main thread of work that starts our whole program and manages the user interface, we normally prefer to think of our work in terms of *queues*.

Queues work like they do in real life, where you might line up to buy something at a grocery store: you join the queue at the back, then move forward again and again until you're at the front, at which point you can check out. Some bigger stores might have lots of queues leading up to lots of checkouts, and small stores might just have one queue with one checkout. You might occasionally see stores trying to avoid the problem of one queue moving faster than another by having a single shared queue feed into multiple checkouts – there are all sorts of possible combinations.

Swift's queues work exactly the same way: we create a queue and add work to it, and the system will remove and execute work from there in the order it was added. Sometimes the queues are *serial*, which means they remove one piece of work from the front of the queue and complete it before going onto the next piece of work; and sometimes they are *concurrent*, which means they remove and execute multiple pieces of work at a time. Either way work will start in the order it was added to the queue unless we specifically say something has a high or low priority.

You might look at that and wonder why you even need serial queues – surely running one thing at a time is what we're trying to avoid? Well, no. In fact, there are lots of times when having the predictability of a serial queue is important.

As a simple example, your user might want to batch convert a collection of videos from one format to another. Video conversion is a really intense operation and is already highly optimized to take full advantage of multi-core CPUs, so it's more efficient to convert one video fully, then the next, then a third, and so on until you reach the end, rather than trying to convert four at once. This kind of work is perfect for a serial queue.

More importantly, sometimes serial queues are *required* to ensure our data is safe. For example, manipulating your user's data is exactly the kind of work you'd want to do on a serial queue, because it stops you from trying to read the data at the same time some other part of your program is trying to write new data.

Putting this all together, threads are the individual slices of a program that do pieces of work, whereas queues are like pipelines of execution where we can request that work be done at some point.

Queues are easier to think about than threads because they focus on what matters: we don't care how some code runs on the CPU, as long as it either runs in a particular order (serially) or not (concurrently). A lot of the time we don't even create the queue – we just use one of the built-in queues and let the system figure out how it should happen.

Tip: Sometimes Apple's frameworks will help you a little here. For example, even though using the `@State` property wrapper in a view will cause the body to be refreshed when the property is changed, this property wrapper is designed to be safe to call on any thread.

5. Main thread and main queue whats the difference

The main thread is the one that starts our program, and it's also the one where all our UI work must happen. However, there is also a main *queue*, and although sometimes we use the terms "main thread" and "main queue" interchangeably, they aren't quite the same thing.

It's a subtle distinction, but it can sometimes matter: although your main queue will always execute on the main thread (and is therefore where you'll be doing your UI work!), it's also possible that other queues might sometimes run on the main thread – the system is free to move things around in whatever way is most efficient.

So, if you're on the main queue then you're definitely on the main thread, but being on the main thread doesn't automatically mean you're on the main queue – a different queue could temporarily be running on the main thread.

At this point you're very likely staring at the screen wondering when this would ever be a problem, or perhaps even rereading what I said like it's a cryptic riddle. Trust me, if you ever hit a problem where the main thread vs main queue matters, you'll be glad you read this!

6. Where is swift concurrency supported

Swift concurrency is supported from iOS 13, macOS 10.15, watchOS 6, tvOS 13, and visionOS 1.0. This offers the full range of Swift functionality, including actors, async/await, the task APIs, and more.

Important: This backwards compatibility applies only to Swift language features, not to any APIs built using those language features. This means you can write your own code to use async/await, actors, and so on, but you won't automatically gain access to newer APIs using those.

If you are keen to use newer APIs in your project while also preserving backwards compatibility for older OS releases, your best bet is to add a runtime version check for whatever your target version is, then wrap the older APIs with continuations. This kind of hybrid solution allows you to keep using async/await elsewhere in your project – you get all the benefits of concurrency for the vast majority of your code, while keeping your backwards deployment shims neatly organized in one place so they can be removed in a year or two.

7. Dedication

This book is dedicated to the many people who helped read through earlier editions, reporting typos, suggesting improvements, and helping make this book as comprehensive as it can be – I'm really grateful they took the time to provide so much help and support!

In alphabetical order, they are: Abdul Ajetunmobi, Alejandro Martinez, Andrew Cowley, Chad Naeger, Chris Rivers, Darius Dunlap, Devanshi Modha, Diego Freniche Brito, Eric Schofield, Glenn Sequeira, Gustavo Diel, Hamish Allan, Horacio Chavez, Howard Katz, Isa Hashim, Jana Grill, Jason Bruder, Jeff Terry, Lars Christiansen, Laurent Brusa, Tor Rafsol Løseth, Mats Braa, Michael M. Mayer, Paul Williamson, Phil Martin, Piers Ebdon, Rick Gigger, Sarah Reichelt, Thomas Alvarez, Tom Comer, Vikash Anand, and Zack Apiratitham.

8. What is a synchronous function

By default, all Swift functions are *synchronous*, but what does that mean? A synchronous function is one that executes all its work in a simple, straight line on a single thread. Although the function itself can interact with other threads – it can request that work happens elsewhere, for example – the function itself always runs on a single thread.

So, code like this represents a series of synchronous functions

```
func functionA() {
    var counter = 0
    functionB()
    // etc
}

func functionB() {
    functionC()
    // etc
}

func functionC() {
    var counter = 0
    // etc
}
```

This has a number of advantages, not least that synchronous functions are very easy to think about: when you call function A, it will carry on working until all its work is done, then return a value. As part of that work, function A calls function B, which perhaps functions C, D, and E as well, and it doesn't matter – they all will execute on the same thread, and run one by one until the work completes.

Internally this is handled as a function stack: whenever one function calls another, the system creates what's called a *stack frame* to store all the data required for that new function – that's things like its local variables, for example. The stack frame for function A will store its local variable, for example.xxxx

That new stack frame gets pushed on top of the previous one, like a stack of Lego bricks, and if that function calls a third function then another stack frame is created and added above the others. Eventually the functions finish, and their stack frame is removed and destroyed in a process we call *popping*, and control goes back to whichever function the code was called from.

This approach means that both function A and C have a variable called `counter`, but they don't clash – Swift tracks them independently.

Synchronous functions have an important downside, which is that they are *blocking*. If function A calls function B and needs to know what its return value is, then function A must wait for function B to finish before it can continue doing its work.

This sounds self-evident, I know, but blocking code is problematic because now you've blocked a whole thread – it might be sitting around for a second or more waiting for some data to return.

You're probably thinking that waiting for a second is nothing at all, but in computing terms that's an ice age! Keep in mind that the Neural Engine in Apple's M4 CPU – just one part of the chip that powers a modern Mac – is capable of doing 38 *trillion* things per second, so if you block a thread for even a second that's 38,000,000,000,000 things you could have done but didn't.

One solution is to say "Well, if we've got a thread that's currently blocked, we should just launch a new thread." That's definitely *a* solution, but it's also a path towards thread explosion where the system creates so many threads it becomes a burden.

So, although synchronous functions are easy to think about and work with, they aren't very efficient for certain kinds of tasks. To make our code more flexible and more efficient, it's possible to create *asynchronous* functions instead.

9. What is an asynchronous function

Although Swift functions are synchronous by default, we can make them *asynchronous* by adding one keyword: `async`. Inside asynchronous functions, we can call other asynchronous functions using a second keyword: `await`. As a result, you'll often hear Swift developers talk about async/await as a way of coding.

So, this is a synchronous function that rolls a virtual dice and returns its result:

```
func randomD6() -> Int {
    Int.random(in: 1...6)
}

let result = randomD6()
print(result)
```

Download this as an Xcode project

And this is an *asynchronous* or *async* function:

```
func randomD6() async -> Int {
    Int.random(in: 1...6)
}

let result = await randomD6()
print(result)
```

Download this as an Xcode project

The only part of the code that changed is adding the `async` keyword before the return type and the `await` keyword before calling it, but those changes tell us three important things about async functions.

First, `async` is part of the function's *type*. The original, synchronous function returns an integer, which means we can't use it in a place that expects it to return a string. However, by marking the code `async` we've now made it an *asynchronous* function that returns an integer, which means we can't use it in a place that expects a *synchronous* function that returns an integer.

This is what I mean when I say that the async nature of the function is part of its type: it affects the way we refer to the function everywhere else in our code. This is exactly how `throws` works – you can't use a throwing function in a place that expects a non-throwing function.

Second, notice that the work inside our function hasn't actually changed. The same work is being done as before: this function doesn't actually use the `await` keyword at all, and that's okay. You see, marking a function with `async` means it *might* do asynchronous work, not that it must. Again, the same is true of `throws`: some paths through a function might throw errors, but others might not.

A third key difference arises when we *call* `randomD6()`, because we need to do so asynchronously. Swift provides a few ways we can do this, but in our example we used `await`, which means "run this function asynchronously and wait for its result to come back before continuing."

So, what's the *actual difference* between synchronous and asynchronous functions? To answer that, I want to show you a real function that does some async work to fetch a file from a web server:

```
func fetchNews() async -> Data? {
    do {
        let url = URL(string: "https://hws.dev/news-1.json")!
        let (data, _) = try await URLSession.shared.data(from: url)
        return data
    } catch {
        print("Failed to fetch data")
        return nil
    }
}

if let data = await fetchNews() {
    print("Downloaded \(data.count) bytes")
```

```
} else {
    print("Download failed.")
}
```

Download this as an Xcode project

Later on we'll be digging in more to how that actually works, but for now what matters is that the `URLSession.shared.data(from:)` method we call is asynchronous – its job is to fetch some data from a web server, without causing the whole program to freeze up.

We've already seen that synchronous functions cause *blocking*, which leads to performance problems. Async functions do *not* block: when we call them with `await` we are marking a suspension point, which is a place where the function can suspend itself – literally stop running – so that other work can happen. At some point in the future the function's work completes, and Swift will wake it back up out of its "suspended animation"-like existence and it will carry on working.

This might sound simple on paper, but in practice it's very clever.

First, when an async function is suspended, all the async functions that called it are also suspended; they all wait quietly while the async work happens, then resume later on. This is really important: async functions have this special ability to be suspended that regular synchronous functions do not. It's for this reason that synchronous functions cannot call async functions directly – they don't know how to suspend themselves.

Second, a function can be suspended as many times as is needed, but it won't happen without you writing `await` there – functions won't suspend themselves by surprise.

Third, a function that is suspended does *not* block the thread it's running on, and instead it gives up that thread so that Swift can do other work instead.

Although we can tell Swift how important many tasks are, we don't get to decide exactly how the system schedules our work – it automatically takes care of all the threads working under the hood. This means if we call async function A without waiting for its result, then a moment later call async function B, it's entirely possible B will start running before A does.

Fourth, when the function resumes, it might be running on the same thread as before, but it might not; Swift gets to choose, and you shouldn't make any assumptions here. This means by the time your function resumes all sorts of things might have changed in your program – a few milliseconds might have passed, or perhaps 20 seconds or more.

And finally, I know I'm repeating myself, but this matters: just because a function is async doesn't mean it *will* suspend – the `await` keyword only marks a *potential suspension point*.

Swift knows perfectly well that the function we're calling is async, so this `await` keyword is a marker for us humans rather than for the compiler – it's a way of clearly marking which parts of the function might suspend, so you can know for sure which parts of the function run as one atomic chunk. ("Atomic" is a fancy word meaning "indivisible" – a chunk of work where all lines of code will execute without being interrupted by other code running.) This requirement for `await` is identical to the requirement for `try`, where we must mark each line of code that might throw errors.

So, async functions are like regular functions, except they have a superpower: if they need to, they can suspend themselves and all their callers, freeing up their thread to do other work.

10. How to create and call an async function

Using async functions in Swift is done in two steps: declaring the function itself as being `async`, then calling that function using `await`.

For example, if we were building an app that wanted to download a whole bunch of temperature readings from a weather station, calculate the average temperature, then upload those results, then we might want to make all three of those async:

To actually *use* those functions we would then need to write a fourth function that calls them one by one and prints the response. This function also needs to be async, because in theory the three functions it calls could suspend and so it might also need to be suspended.

I'm not going to do the actual networking code here, because we'll be looking a lot at networking later on. Instead, I want to focus on the structure of our functions so you can see how they fit together, so we'll be using mock data here – random numbers for the weather data, and the string "OK" for our server response.

Here's the code:

```
func fetchWeatherHistory() async -> [Double] {
    (1...100_000).map { _ in Double.random(in: -10...30) }
}

func calculateAverageTemperature(for records: [Double]) async -> Double {
    let total = records.reduce(0, +)
    let average = total / Double(records.count)
    return average
}

func upload(result: Double) async -> String {
    "OK"
}

func processWeather() async {
    let records = await fetchWeatherHistory()
    let average = await calculateAverageTemperature(for: records)
    let response = await upload(result: average)
    print("Server response: \(response)")
}

await processWeather()
```

Download this as an Xcode project

So, we have three simple async functions that fit together to form a sequence: download some data, process that data, then upload the result. That all gets stitched together into a cohesive flow using the `processWeather()` function, which can then be called from elsewhere.

That's not a lot of code, but it *is* a lot of functionality:

- Every one of those `await` calls is a potential suspension point, which is why we marked it explicitly. Like I said, one async function can suspend as many times as is needed.
- Swift will run each of the `await` calls in sequence, waiting for the previous one to complete. This is *not* going to run several things in parallel.
- Each time an `await` call finishes, its final value gets assigned to one of our constants – `records`, `average`, and `response`. Once created this is just regular data, no different from if we had created it synchronously.
- Because it calls async functions using `await`, it is *required* that `processWeather()` be itself an async function. If you remove that Swift will refuse to build your code.

When reading async functions like this one, it's good practice to look for the `await` calls because they are all places where unknown other amounts of work might take place before the next line of code executes.

Think of it a bit like this:

```swift
func processWeather() async {
    let records = await fetchWeatherHistory()
    // anything could happen here
    let average = await calculateAverageTemperature(for: records)
    // or here
    let response = await upload(result: average)
    // or here
    print("Server response: \(response)")
}
```

We're only using local variables inside this function, so they are safe. However, if you were relying on properties from a class, for example, they might have changed between each of those `await` lines.

11. How to call async throwing functions

Just like their synchronous counterparts, Swift's async functions can be throwing or non-throwing depending on how you want them to behave. However, there is a twist: although we mark the function as being `async throws`, we *call* the function using `try await` – the keyword order is flipped.

To demonstrate this in action, here's a `fetchFavorites()` method that attempts to download some JSON from my server, decode it into an array of integers, and return the result:

```
func fetchFavorites() async throws -> [Int] {
    let url = URL(string: "https://hws.dev/user-favorites.json")!
    let (data, _) = try await URLSession.shared.data(from: url)
    return try JSONDecoder().decode([Int].self, from: data)
}

if let favorites = try? await fetchFavorites() {
    print("Fetched \(favorites.count) favorites.")
} else {
    print("Failed to fetch favorites.")
}
```

Download this as an Xcode project

Both fetching data and decoding it are throwing functions, so we need to use `try` in both those places. Those errors aren't being handled in the function, so we need to mark `fetchFavorites()` as also being throwing so Swift can let any errors bubble up to whatever called it.

Notice that the function is marked `async throws` but the function *calls* are marked `try await` – like I said, the keyword order gets reversed. So, it's "asynchronous, throwing" in the function definition, but "throwing, asynchronous" at the call site. Think of it as unwinding a stack.

Not only does `try await` read more easily than `await try`, but it's also more reflective of what's actually happening when our code executes: we're waiting for some work to complete, and when it does complete we'll check whether it ended up throwing an error or not.

Of course, the other possibility is that they could have `try await` at the call site and `throws async` on the function definition, and here's what the Swift team says about that:

> *"This order restriction is arbitrary, but it's not harmful, and it eliminates the potential for stylistic debates."*

Personally, if I saw `throws async` I'd be more likely to write it as "this thing throws an async error" or similar, but as the quote above says ultimately the order is just arbitrary.

12. What calls the first async function

You can only call async functions from other async functions, because they might need to suspend themselves and everything that is waiting for them. This leads to a bit of a chicken and egg problem: if only async functions can call other async functions, what starts it all – what calls the very first async function?

Well, there are three main approaches you'll find yourself using.

First, in simple command-line programs using the `@main` attribute, you can declare your `main()` method to be async. This means your program will immediately launch into an async function, so you can call other async functions freely.

Here's how that looks in code:

```
func processWeather() async {
    // Do async work here
}

@main
struct MainApp {
    static func main() async {
        await processWeather()
    }
}
```

Download this as an Xcode project

Second, in apps built with something like SwiftUI the framework itself has various places that can trigger an async function. For example, the `refreshable()` and `task()` modifiers can both call async functions freely.

Using the `task()` modifier we could write a simple "View Source" app for iOS that fetches the content of a website when our view appears:

```
struct ContentView: View {
    @State private var sourceCode = ""

    var body: some View {
        ScrollView {
            Text(sourceCode)
        }
        .task {
            await fetchSource()
        }
    }

    func fetchSource() async {
        do {
            let url = URL(string: "https://apple.com")!

            let (data, _) = try await URLSession.shared.data(from: url)
            sourceCode = String(decoding: data, as: UTF8.self).trimmingCharacters(in: .whitespacesAndNewlines)
        } catch {
            sourceCode = "Failed to fetch apple.com"
        }
    }
}
```

Download this as an Xcode project

Tip: When do async work you might end up pushing work away from the main thread where UI updates must happen, but the `@State` property wrapper has specifically been written to allow us to modify its value on any thread.

The third option is that Swift provides a dedicated `Task` API that lets us call async functions from a synchronous function. Now, you might think "wait a minute – how can a synchronous function call an asynchronous function?"

Well, it *can't* – at least not *directly*. Remember, async functions might need to suspend themselves in the future, and synchronous functions don't know how to do that.

When you use something like `Task` you're asking Swift to run some async code. If you don't care about the result you have nothing to wait *for* – the task will start running immediately while your own function continues, and it will always run

to completion even if you don't store the active task somewhere.

This means you're not awaiting the result of the task, so you won't run the risk of being suspended. Of course, when you actually want to *use* any returned value from your task, that's when `await` is required.

We'll be looking at Swift's `Task` API in detail later on, but for now we could quickly upgrade our little website source code viewer to work with any URL.

This time we're going to trigger the network fetch using a button press, which is *not* asynchronous by default, so we're going to wrap our work in a `Task`. This is possible because we don't need to wait for the task to complete – it will always run to completion as soon as it is made, and will take care of updating the UI for us.

Here's how that looks:

```
struct ContentView: View {
    @State private var site = "https://"
    @State private var sourceCode = ""

    var body: some View {
        VStack {
            HStack {
                TextField("Website address", text: $site)
                    .textFieldStyle(.roundedBorder)
                Button("Go") {
                    Task {
                        await fetchSource()
                    }
                }
            }
            .padding()

            ScrollView {
                Text(sourceCode)
            }
        }
    }

    func fetchSource() async {
        do {
            let url = URL(string: site)!
            let (data, _) = try await URLSession.shared.data(from: url)
            sourceCode = String(decoding: data, as: UTF8.self).trimmingCharacters(in: .whitespacesAndNewlines)
        } catch {
            sourceCode = "Failed to fetch \(site)"
        }
    }
}
```

Download this as an Xcode project

It's remarkable how we can accomplish so much with such little code!

13. Whats the performance cost of calling an async function

Whenever we use `await` to call an async function, we mark a potential suspension point in our code – we're acknowledging that it's entirely possible our function will be suspended, along with all its callers, while the work completes.

You can see the suspension chain in action with code like this:

```
func countFavorites() async throws {
    let favorites = try await decodeFavorites()
    print("Downloaded \(favorites.count) favorites.")
}

func decodeFavorites() async throws -> [Int] {
    let data = try await loadFavorites()
    return try JSONDecoder().decode([Int].self, from: data)
}

func loadFavorites() async throws -> Data {
    let url = URL(string: "https://hws.dev/user-favorites.json")!
    let (data, _) = try await URLSession.shared.data(from: url)
    return data
}
```

When that code runs, `countFavorites()` will call `decodeFavorites()`, which in turn will call `loadFavorites()`. Inside `loadFavorites()` is our actual networking code, which will suspend while the work happens, and in turn cause all three functions – `loadFavorites()`, `decodeFavorites()`, and `countFavorites()` – to suspend.

In terms of performance, this is *not* free: synchronous and asynchronous functions use a different calling convention internally, with the asynchronous variant being slightly less efficient.

The important thing to understand here is that Swift cannot tell at compile time whether an `await` call will suspend or not, and so the same (slightly) more expensive calling convention is used regardless of what actually takes place at runtime.

With our networking code, for example, it's entirely possible the system chooses not to suspend if the data is already cached from a previous load, or if the network connection is offline – we don't know, and neither does Swift until the code actually runs.

What happens at runtime depends on whether the call suspends or not:

- If a suspension happens, then Swift will pause the function and all its callers, which has a small performance cost. These will then be resumed later, and ultimately whatever performance cost you pay for the suspension is like a rounding error compared to the performance gain provided by async/await even existing.
- If a suspension does *not* happen, no pause will take place and your function will continue to run with the same efficiency and timings as a synchronous function.

That last part carries an important side effect: using `await` will not cause your code to wait for one runloop to go by before continuing.

It's a common joke that many coding problems can be fixed by waiting for one runloop tick to pass before trying again – usually seen as `DispatchQueue.main.async { … }` in Swift projects – but that will *not* happen when using `await`, because the code will execute immediately.

So, if your code doesn't actually suspend, the only cost to calling an asynchronous function is the slightly more expensive calling convention, and if your code *does* suspend then any cost is more or less irrelevant because you've gained so much extra performance thanks to the suspension happening in the first place.

14. How to create and use async properties

Just as Swift's functions can be asynchronous, computed properties can also be asynchronous: attempting to access them must also use `await` or similar, and may also need `throws` if errors can be thrown when computing the property.

This is what allows things like the `value` property of `Task` to work – it's a simple property, but we must access it using `await` because it might not have completed yet. If you look at the property in Xcode's generated interface for `Task`, you'll see it's declared as this:

```
public var value: Success { get async throws }
```

That means it has a getter than runs asynchronously and can throw errors.

Important: This is only possible on read-only computed properties – attempting to provide a setter will cause a compile error.

To demonstrate this, we could create a `RemoteFile` struct that stores a URL and a type that conforms to `Decodable`. This struct won't actually fetch the URL when the struct is created, but will instead dynamically fetch the contents of the URL every time the property is requested so that we can update our UI dynamically.

Tip: If you use `URLSession.shared` to fetch your data it will automatically be cached, so we're going to create a custom URL session that always ignores local and remote caches to make sure our remote file is always fetched.

Here's the code:

```
// First, a URLSession instance that never uses caches
extension URLSession {
    static let noCacheSession: URLSession = {
        let config = URLSessionConfiguration.default
        config.requestCachePolicy = .reloadIgnoringLocalAndRemoteCacheData
        return URLSession(configuration: config)
    }()
}

// Now our struct that will fetch and decode a URL every
// time we read its `contents` property
struct RemoteFile<T: Decodable> {
    let url: URL
    let type: T.Type

    var contents: T {
        get async throws {
            let (data, _) = try await URLSession.noCacheSession.data(from: url)
            return try JSONDecoder().decode(T.self, from: data)
        }
    }
}
```

So, we're fetching the URL's contents every time `contents` is accessed, as opposed to storing the URL's contents when a `RemoteFile` instance is created. As a result, the property is marked both `async` and `throws` so that callers must use `await` or similar when accessing it.

To try that out with some real SwiftUI code, we could write a view that fetches messages. We don't ever want stale data, so we're going to point our `RemoteFile` struct at a particular URL and tell it to expect an array of `Message` objects to come back, then let it take care of fetching and decoding those while also bypassing the `URLSession` cache:

```
// First, a URLSession instance that never uses caches
extension URLSession {
    static let noCacheSession: URLSession = {
        let config = URLSessionConfiguration.default
        config.requestCachePolicy = .reloadIgnoringLocalAndRemoteCacheData
        return URLSession(configuration: config)
    }()
}

// Now our struct that will fetch and decode a URL every
// time we read its `contents` property
```

```
struct RemoteFile<T: Decodable> {
    let url: URL
    let type: T.Type

    var contents: T {
        get async throws {
            let (data, _) = try await URLSession.noCacheSession.data(from: url)
            return try JSONDecoder().decode(T.self, from: data)
        }
    }
}

struct Message: Decodable, Identifiable {
    let id: Int
    var user: String
    var text: String
}

struct ContentView: View {
    let source = RemoteFile(url: URL(string: "https://hws.dev/inbox.json")!, type: [Message].self)
    @State private var messages = [Message]()

    var body: some View {
        NavigationStack {
            List(messages) { message in
                VStack(alignment: .leading) {
                    Text(message.user)
                        .font(.headline)
                    Text(message.text)
                }
            }
            .navigationTitle("Inbox")
            .toolbar {
                Button("Refresh", systemImage: "arrow.clockwise", action: refresh)
            }
            .onAppear(perform: refresh)
        }
    }

    func refresh() {
        Task {
            do {
                // Access the property asynchronously
                messages = try await source.contents
            } catch {
                print("Message update failed.")
            }
        }
    }
}
```

Download this as an Xcode project

That call to `source.contents` is where the real action happens – it's a property, yes, but it must also be accessed asynchronously so that it can do its work of fetching and decoding without blocking the UI.

15. How to call an async function using async let

Sometimes you want to run several async operations at the same time then wait for their results to come back, and the easiest way to do that is with `async let`. This lets you start several async functions, all of which begin running immediately – it's much more efficient than running them sequentially.

A common example of where this is useful is when you have to make two or more network requests, none of which relate to each other. That is, if you need to get Thing X and Thing Y from a server, but you don't need to wait for X to return before you start fetching Y.

To demonstrate this, we could define a couple of structs to store data – one to store a user's account data, and one to store all the messages in their inbox:

```
struct User: Decodable, Identifiable {
    let id: UUID
    let name: String
    let age: Int
}

struct Message: Decodable, Identifiable {
    let id: Int
    let from: String
    let message: String
}
```

These two things can be fetched independently of each other, so rather than fetching the user's account details *then* fetching their message inbox we want to get them both together.

In this instance, rather than using a regular `await` call a better choice is `async let`, like this:

```
func loadData() async {
    async let (userData, _) = URLSession.shared.data(from: URL(string: "https://hws.dev/user-24601.json")!)

    async let (messageData, _) = URLSession.shared.data(from: URL(string: "https://hws.dev/user-messages.json")!)

    // more code to come
}
```

That's only a small amount of code, but there are three things I want to highlight in there:

- Even though the `data(from:)` method is async, we don't need to use `await` before it because that's implied by `async let`.
- The `data(from:)` method is also throwing, but we don't need to use `try` to execute it because that gets pushed back to when we actually want to read its return value.
- Both those network calls start immediately, but might complete in any order.

Okay, so now we have two network requests in flight. The next step is to wait for them to complete, decode their returned data into structs, and use that somehow.

There are two things you need to remember:

- Both our `data(from:)` calls might throw, so when we read those values we need to use `try`.
- Both our `data(from:)` calls are running concurrently while our main `loadData()` function continues to execute, so we need to read their values using `await` in case they aren't ready yet.

So, we could complete our function by using `try await` for each of our network requests in turn, then print out the result:

```
struct User: Decodable, Identifiable {
    let id: UUID
    let name: String
    let age: Int
}

struct Message: Decodable, Identifiable {
```

```
    let id: Int
    let from: String
    let message: String
}

func loadData() async {
    async let (userData, _) = URLSession.shared.data(from: URL(string: "https://hws.dev/user-24601.json")!)

    async let (messageData, _) = URLSession.shared.data(from: URL(string: "https://hws.dev/user-messages.json")!)

    do {
        let decoder = JSONDecoder()
        let user = try await decoder.decode(User.self, from: userData)
        let messages = try await decoder.decode([Message].self, from: messageData)
        print("User \(user.name) has \(messages.count) message(s).")
    } catch {
        print("Sorry, there was a network problem.")
    }
}

await loadData()
```

Download this as an Xcode project

The Swift compiler will automatically track which `async let` constants could throw errors and will enforce the use of `try` when reading their value. It doesn't matter which form of `try` you use, so you can use `try`, `try?` or `try!` as appropriate.

Tip: If you never try to read the value of a throwing `async let` call – i.e., if you've started the work but don't care what it returns – then you don't need to use `try` at all, which in turn means the function running the `async let` code might not need to handle errors at all.

Although both our network requests are happening at the same time, we still need to wait for them to complete in some sort of order. So, if you wanted to update your user interface as soon as either `user` or `messages` arrived back `async let` isn't going to help by itself – you should look at the dedicated `Task` type instead.

One complexity with `async let` is that it captures any values it uses, which means you might accidentally try to write code that isn't safe. Swift helps here by taking some steps to enforce that you aren't trying to modify data unsafely.

As an example, if we wanted to fetch the favorites for a user, we might have a function such as this one:

```
struct User: Decodable, Identifiable {
    let id: UUID
    let name: String
    let age: Int
}

struct Message: Decodable, Identifiable {
    let id: Int
    let from: String
    let message: String
}

func fetchFavorites(for user: User) async -> [Int] {
    print("Fetching favorites for \(user.name)…")

    do {
        async let (favorites, _) = URLSession.shared.data(from: URL(string: "https://hws.dev/user-favorites.json")!
        return try await JSONDecoder().decode([Int].self, from: favorites)
    } catch {
        return []
    }
}

let user = User(id: UUID(), name: "Taylor Swift", age: 26)
async let favorites = fetchFavorites(for: user)
await print("Found \(favorites.count) favorites.")
```

Download this as an Xcode project

That function accepts a `User` parameter so it can print a status message. But what happens if our `User` struct was created as a class instead? You can see this for yourself if you change the struct to this:

```
class User: Decodable, Identifiable {
    let id: UUID
    let name: String
    let age: Int
```

```
    init(id: UUID, name: String, age: Int) {
        self.id = id
        self.name = name
        self.age = age
    }
}
```

With that class in place, Swift will issue a rather opaque error message: "Non-sendable type 'User' in implicitly asynchronous access to main actor-isolated let 'user' cannot cross actor boundary."

What it means is that we're implicitly sending our `User` instance somewhere else. Behind the scenes, `async let` works by making an implicit asynchronous task for us; it's shorthand syntactic sugar to make this code easier to write. Calling `fetchFavorites()` takes place in that implicit task, so even though it's not apparent, our code takes our `User` object and sends it somewhere else – into a different task to do the work.

Swift doesn't think it's safe to do so. This is because class instances are shared, so two different parts of our code might try to write data at the same time.

To fix this, we need to mark the class as `final`, and also tell Swift it's safe to be sent across different tasks, like this:

```
final class User: Decodable, Identifiable, Sendable {
```

Swift will now validate at compile time that the class is safe to send, so make those two small changes is all it takes to make our code work.

`Sendable` is quite a tricky topic to understand, so I'll cover it in detail in the very next chapter.

16. Sending data safely across actor boundaries

Swift tries to ensure access to shared data is done safely, partly through types such as actors, and partly through a concept of *sendability* implemented through the `Sendable` protocol and the `@Sendable` attribute. You'll know you've hit this problem when you see the error, "Non-sendable type 'YourType' returned by implicitly asynchronous call to nonisolated function cannot cross actor boundary."

It's quite a dense error, so let me simplify it for you: "you're making an object here, and you want to use it somewhere else, but that isn't safe."

You can see the error yourself with code like this:

```
class User {
    let name: String
    let password: String

    init(name: String, password: String) {
        self.name = name
        self.password = password
    }
}

@main
struct App {
    static func main() async {
        async let user = User(name: "twostraws", password: "fr0stles")
        await print(user.name)
    }
}
```

You can see that we have implicitly made a class be accessed on a separate task through our use of `async let`, but you can also see that both the class's properties are marked as constant, so actually it's safe.

To fix the problem, we need to mark the class as being safe to send across different tasks in our app. This is done in two steps, starting with making the class conform to the `Sendable` protocol, like this:

```
class User: Sendable {
```

That tells Swift this class can be sent between tasks safely, which Swift validates for us – it will make sure all the properties are also `Sendable`, otherwise it will refuse to build. In this case, the properties are constant value types, so they are safe to share.

And secondly, we need to declare the class as being `final` – saying that it can't be subclassed – like this:

```
final class User: Decodable, Sendable {
```

The `final` part here is important: Swift can't guarantee this class is definitely sendable unless it can't be subclassed, just in case we make a subclass that adds a property that *isn't* sendable, at which point things get rather messy.

So, the completed code is this:

```
final class User: Decodable, Sendable {
    let name: String
    let password: String

    init(name: String, password: String) {
        self.name = name
        self.password = password
    }
}

@main
struct App {
    static func main() async {
        async let user = User(name: "twostraws", password: "fr0stles")
```

```
            await print(user.name)
        }
    }
```

Download this as an Xcode project

Swift automatically considers all actors as conforming to `Sendable` because they automatically handle synchronization correctly. It will also automatically consider structs and enums as being `Sendable`, as long as they only contain values that are also `Sendable`.

Things are a little more complex when it comes to functions, because Swift needs to make sure the function or the type that *owns* the function can also be sent correctly.

Let's say we wanted to make a tiny wrapper around the old `asyncAfter()` method from GCD, so we can execute a function after a short delay. We'd need to write code like this:

```
func runLater(_ function: @escaping @Sendable () -> Void) {
    DispatchQueue.global().asyncAfter(deadline: .now() + 3, execute: function)
}
```

This is because `asyncAfter()` only accepts functions that can safely be sent between tasks, to avoid data races. So, we need to pass that requirement up to whoever calls `runLater()`, which is what the `@Sendable` attribute is doing – we're saying whatever function we're told to run must be safe to transfer.

The same is true for a great many other APIs that need to run their work elsewhere. For example, making a `Timer` fire repeatedly to run an action again and again requires an `@Sendable @escaping` function to run:

```
func repeatAction(_ action: @escaping @Sendable () -> Void) {
    Timer.scheduledTimer(withTimeInterval: 1, repeats: true) { _ in
        action()
    }
}
```

To put that to use, we could make a simple `Counter` actor that was able to increment its value whenever we wanted. Actors are inherently `Sendable`, so we can create a task to call our counter's `increment()` method and use that safely with `repeatAction()`:

```
func repeatAction(_ action: @escaping @Sendable () -> Void) {
    Timer.scheduledTimer(withTimeInterval: 1, repeats: true) { _ in
        action()
    }
}

actor Counter {
    var value = 0

    func increment() {
        value += 1
        print("Counter incremented: \(value)")
    }
}

let counter = Counter()

repeatAction {
    Task {
        // This is safe
        await counter.increment()
    }
}

// Make sure the timer has chance to fire a few times.
try? await Task.sleep(for: .seconds(5))
```

Download this as an Xcode project

So, the `Task` is `Sendable` if its work is also `Sendable`.

17. Whats the difference between await and async let

Swift lets us perform async operations using both `await` and `async let`, but although they both run some async code they don't quite run the same: `await` waits for the work to complete so we can read its result, whereas `async let` does not.

Which you choose depends on the kind of work you're trying to do. For example, if you want to make two network requests where one relates to the other, you might have code like this:

```
func fetchData() async -> Data? {
    // do work here
    nil
}

func process(_ data: Data?) async -> Bool {
    true
}

let download = await fetchData()
let result = await process(download)
```

There the call to `process()` cannot start until the call to `fetchData()` has completed and returned its value – it just doesn't make sense for those two to run simultaneously.

On the other hand, if you're making several completely different requests – perhaps you want to download the latest news, the weather forecast, and check whether an app update was available – then those things do *not* rely on each other to complete and would be great candidates for `async let`:

```
func getNews() async -> [String] { [] }
func getWeather() async -> [String] { [] }
func getUpdateAvailable() async -> Bool { true }

func getAppData() async -> ([String], [String], Bool) {
    async let news = getNews()
    async let weather = getWeather()
    async let hasUpdate = getUpdateAvailable()
    return await (news, weather, hasUpdate)
}
```

So, use `await` when it's important you have a value before continuing, and `async let` when your work can continue without the value for the time being – you can always use `await` later on when it's actually needed.

In all these things, remember that "great candidates for `async let`" doesn't mean "this should definitely use `async let`." For example, will making multiple simultaneous requests clog up your user's network session? If you're showing information incrementally – i.e., if you use the result of each network request as soon as it finishes – then running multiple requests at the same time might make each one complete more slowly.

Similarly, do any of the requests you're making take time to execute on the server? If so, firing off lots of requests is likely to work well because they can all happen in parallel, particularly if they are going to different servers.

18. Why cant we call async functions using async var

Swift's `async let` syntax provides short, helpful syntax for running lots of work concurrently, allowing us to wait for them all later on. However, it only works as `async let` – it's not possible to use `async var`.

If you think about it, this restriction makes sense. Consider pseudocode like this:

```
func fetchUsername() async -> String {
    // complex networking here
    "Taylor Swift"
}

async var username = fetchUsername()
username = "Justin Bieber"
print("Username is \(username)")
```

That attempts to create a variable asynchronously, then writes to it directly. Have we cancelled the async work? If not, when the async work completes will it overwrite our new value? Do we still need to use `await` when reading the value even after we've explicitly set it?

This kind of code would create all sorts of confusion, so it's just not allowed – `async let` is our only option.

19. How to use continuations to convert completion handlers into async functions

Older Swift code uses completion handlers for notifying us when some work has completed, and sooner or later you're going to have to use it from an `async` function – either because you're using a library someone else created, or because it's one of your own functions but updating it to async would take a lot of work.

Swift uses *continuations* to solve this problem, allowing us to create a bridge between older functions with completion handlers and newer async code.

To demonstrate this problem, here's some code that attempts to fetch some JSON from a web server, decode it into an array of `Message` structs, then send it back using a completion handler:

```
struct Message: Decodable, Identifiable {
    let id: Int
    let from: String
    let message: String
}

func fetchMessages(completion: @Sendable @escaping ([Message]) -> Void) {
    let url = URL(string: "https://hws.dev/user-messages.json")!

    URLSession.shared.dataTask(with: url) { data, response, error in
        if let data {
            if let messages = try? JSONDecoder().decode([Message].self, from: data) {
                completion(messages)
                return
            }
        }

        completion([])
    }.resume()
}
```

Although the `dataTask(with:)` method does run our code on its own thread, this is *not* an async function in the sense of Swift's async/await feature, which means it's going to be messy to integrate into other code that *does* use modern async Swift.

To fix this, Swift provides us with *continuations*, which are special objects we pass into the completion handlers as captured values. Once the completion handler fires, we can either return the finished value, throw an error, or send back a `Result` that can be handled elsewhere. There's no time limit on how long continuations can take to complete, but they *must* complete at some point.

In the case of `fetchMessages()`, we want to write a new async function that calls the original, and in its completion handler we'll return whatever value was sent back:

```
struct Message: Decodable, Identifiable {
    let id: Int
    let from: String
    let message: String
}

func fetchMessages(completion: @Sendable @escaping ([Message]) -> Void) {
    let url = URL(string: "https://hws.dev/user-messages.json")!

    URLSession.shared.dataTask(with: url) { data, response, error in
        if let data {
            if let messages = try? JSONDecoder().decode([Message].self, from: data) {
                completion(messages)
                return
            }
        }

        completion([])
    }.resume()
}

func fetchMessages() async -> [Message] {
    await withCheckedContinuation { continuation in
        fetchMessages { messages in
            continuation.resume(returning: messages)
```

```
        }
    }
}

let messages = await fetchMessages()
print("Downloaded \(messages.count) messages.")
```

Download this as an Xcode project

As you can see, starting a continuation is done using the `withCheckedContinuation()` function, which passes into itself the continuation we need to work with. It's called a "checked" continuation because Swift checks that we're using the continuation correctly, which means abiding by one very simple, very important rule:

Your continuation must be resumed exactly once. Not zero times, and not twice or more times – exactly once. Again, there's no time limit on this happening, but it *must* happen at some point.

If you call the checked continuation twice or more, Swift will cause your program to halt – it will just crash. I realize this sounds bad, but when the alternative is to have some bizarre, unpredictable behavior, crashing doesn't sound so bad.

On the other hand, if you fail to resume the continuation at all, Swift will print out a large warning in your debug log similar to this: "SWIFT TASK CONTINUATION MISUSE: fetchMessages() leaked its continuation!" This is because you're leaving the task suspended, causing any resources it's using to be held indefinitely.

You might think these are easy mistakes to avoid, but in practice they can occur in all sorts of places if you aren't careful.

For example, in our original `fetchMessages()` method we used this:

```
if let data {
    if let messages = try? JSONDecoder().decode([Message].self, from: data) {
        completion(messages)
        return
    }
}

completion([])
```

That checks for data coming back, and checks that it can be decoded correctly, before completing and returning, but if either of those two checks fail then the completion handler is called with an empty array – no matter what happens, the completion handler gets called.

But what if we had written something different? See if you can spot the problem with this alternative:

```
if let data {
    if let messages = try? JSONDecoder().decode([Message].self, from: data) {
        completion(messages)
    }
} else {
    completion([])
}
```

That attempts to decode the JSON into a `Message` array and send back the result using the completion handler, or send back an empty array if nothing came back from the server.

However, it has a mistake that will cause problems with continuations: if some valid data comes back but *can't* be decoded into an array of messages, the completion handler will never be called and our continuation will be leaked.

These two code samples are fairly similar, which shows how important it is to be careful with your continuations. However, if you have checked your code carefully and you're sure it is correct, you can if you want replace the `withCheckedContinuation()` function with a call to `withUnsafeContinuation()`, which works exactly the same way but doesn't add the runtime cost of checking you've used the continuation correctly.

I say you can do this *if you want*, but I'm dubious about the benefit. It's easy to say "I know my code is safe, go for it!" but I'd be wary about moving across to unsafe code unless you had profiled your code using Instruments and were quite sure Swift's extra checks were causing a performance problem.

Note: If you want a continuation that sends back nothing, you can just use `continuation.resume()` with no parameters.

20. How to create continuations that can throw errors

Swift provides `withCheckedContinuation()` and `withUnsafeContinuation()` to let us create continuations that can't throw errors, but if the API you're using *can* throw errors you should use their throwing equivalents: `withCheckedThrowingContinuation()` and `withUnsafeThrowingContinuation()`.

Both of these replacement functions work identically to their non-throwing counterparts, except now you need to catch any errors thrown inside the continuation.

To demonstrate this, here's the same `fetchMessages()` function we used previously, built without async/await in mind:

```
struct Message: Decodable, Identifiable {
    let id: Int
    let from: String
    let message: String
}

func fetchMessages(completion: @Sendable @escaping ([Message]) -> Void) {
    let url = URL(string: "https://hws.dev/user-messages.json")!

    URLSession.shared.dataTask(with: url) { data, response, error in
        if let data = data {
            if let messages = try? JSONDecoder().decode([Message].self, from: data) {
                completion(messages)
                return
            }
        }

        completion([])
    }.resume()
}
```

If we wanted to wrap that using a continuation, we might decide that having zero messages is an error we should throw rather than just sending back an empty array. That thrown error would then need to be handled outside the continuation somehow.

So, first we'd define the errors we want to throw, then we'd write a newer async version of `fetchMessages()` using `withCheckedThrowingContinuation()`, and handling the "no messages" error using whatever code we wanted:

```
struct Message: Decodable, Identifiable {
    let id: Int
    let from: String
    let message: String
}

func fetchMessages(completion: @Sendable @escaping ([Message]) -> Void) {
    let url = URL(string: "https://hws.dev/user-messages.json")!

    URLSession.shared.dataTask(with: url) { data, response, error in
        if let data = data {
            if let messages = try? JSONDecoder().decode([Message].self, from: data) {
                completion(messages)
                return
            }
        }

        completion([])
    }.resume()
}

// An example error we can throw
enum FetchError: Error {
    case noMessages
}

func fetchMessages() async -> [Message] {
    do {
        return try await withCheckedThrowingContinuation { continuation in
            fetchMessages { messages in
                if messages.isEmpty {
                    continuation.resume(throwing: FetchError.noMessages)
                } else {
                    continuation.resume(returning: messages)
                }
            }
        }
```

```
        } catch {
            return [
                Message(id: 1, from: "Tom", message: "Welcome to MySpace! I'm your new friend.")
            ]
        }
    }

    let messages = await fetchMessages()
    print("Downloaded \(messages.count) messages.")
```

Download this as an Xcode project

As you can see, that detects a lack of messages and sends back a welcome message instead, but you could also let the error propagate upwards by removing `do`/`catch` and making the new `fetchMessages()` function throwing.

Tip: Using `withUnsafeThrowingContinuation()` comes with all the same warnings as using `withUnsafeContinuation()` – you should only switch over to it if it's causing a performance problem.

21. How to store continuations to be resumed later

Many of Apple's frameworks report back success or failure using multiple different delegate callback methods rather than completion handlers, which means a simple continuation won't work.

As a simple example, if you were implementing `WKNavigationDelegate` to handle navigating around a `WKWebView` you would implement methods like this:

```
func webView(_ webView: WKWebView, didFinish navigation: WKNavigation!) {
    // our work succeeded
}

func webView(WKWebView, didFail: WKNavigation!, withError: any Error) {
    // our work failed
}
```

So, rather than receiving the result of our work through a single completion closure, we instead get the result in two different places. In this situation we need to do a little more work to create async functions using continuations, because we need to be able to resume the continuation in either method.

To solve this problem you need to know that continuations are just structs with a specific generic type. For example, a checked continuation that succeeds with a string and never throws an error has the type `CheckedContinuation<String, Never>`, and an unchecked continuation that returns an integer array and can throw errors has the type `UnsafeContinuation<[Int], Error>`.

All this is important because to solve our delegate callback problem we need to stash away a continuation in one method – when we trigger some functionality – then resume it from different methods based on whether our code succeeds or fails.

I want to demonstrate this using real code, so we're going to create an `@Observable` class to wrap Core Location, making it easier to request the user's location.

First, add these imports to your code so we can read their location, and also use SwiftUI's `LocationButton` to get standardized UI:

```
import CoreLocation
import CoreLocationUI
```

Second, we're going to create a small part of a `LocationManager` class that has two properties: one for storing a continuation to track whether we have their location coordinate or an error, and one to track an instance of `CLLocationManager` that does the work of finding the user. This also needs a small initializer so the `CLLocationManager` knows to report location updates to us.

Add this class now:

```
@Observable
class LocationManager: NSObject, CLLocationManagerDelegate {
    var locationContinuation: CheckedContinuation<CLLocationCoordinate2D?, any Error>?
    let manager = CLLocationManager()

    override init() {
        super.init()
        manager.delegate = self
    }

    // More code to come
}
```

Third, we need to add an async function that requests the user's location. This needs to be wrapped inside a `withCheckedThrowingContinuation()` call, so that Swift creates a continuation we can stash away and use later. However, because this will be called from the main actor, we need to mark this method as also running on the main actor to avoid data races.

Add this method to the class now:

```
@MainActor
func requestLocation() async throws -> CLLocationCoordinate2D? {
    try await withCheckedThrowingContinuation { continuation in
        locationContinuation = continuation
        manager.requestLocation()
    }
}
```

And finally we need to implement the two methods that might be called after we request the user's location: `didUpdateLocations` will be called if their location was received, and `didFailWithError` otherwise. Both of these need to resume our continuation, with the former sending back the first location coordinate we were given, and the latter throwing whatever error occurred:

Add these last two methods to the class now:

```
func locationManager(_ manager: CLLocationManager, didUpdateLocations locations: [CLLocation]) {
    locationContinuation?.resume(returning: locations.first?.coordinate)
}

func locationManager(_ manager: CLLocationManager, didFailWithError error: any Error) {
    locationContinuation?.resume(throwing: error)
}
```

So, by storing our continuation as a property we're able to resume it in two different places – once where things go to plan, and once where things go wrong for whatever reason. Either way, no matter what happens our continuation resumes exactly once.

At this point our continuation wrapper is complete, so we can use it inside a SwiftUI view. If we put everything together, here's the end result:

```
@Observable
class LocationManager: NSObject, CLLocationManagerDelegate {
    var locationContinuation: CheckedContinuation<CLLocationCoordinate2D?, any Error>?
    let manager = CLLocationManager()

    override init() {
        super.init()
        manager.delegate = self
    }

    @MainActor
    func requestLocation() async throws -> CLLocationCoordinate2D? {
        try await withCheckedThrowingContinuation { continuation in
            locationContinuation = continuation
            manager.requestLocation()
        }
    }

    func locationManager(_ manager: CLLocationManager, didUpdateLocations locations: [CLLocation]) {
        locationContinuation?.resume(returning: locations.first?.coordinate)
    }

    func locationManager(_ manager: CLLocationManager, didFailWithError error: any Error) {
        locationContinuation?.resume(throwing: error)
    }
}

struct ContentView: View {
    @State private var locationManager = LocationManager()

    var body: some View {
        LocationButton {
            Task {
                if let location = try? await locationManager.requestLocation() {
                    print("Location: \(location)")
                } else {
                    print("Location unknown.")
                }
            }
        }
        .frame(height: 44)
        .foregroundStyle(.white)
        .clipShape(.capsule)
        .padding()
```

```
        }
    }
```

Download this as an Xcode project

22. How to fix the error async call in a function that does not support concurrency

This error occurs when you've tried to call an async function from a synchronous function, which is not allowed in Swift – asynchronous functions must be able to suspend themselves and their callers, and synchronous functions simply don't know how to do that.

You can see it with code like this:

```
func doAsyncWork() async {
    print("Doing async work")
}

func doRegularWork() {
    await doAsyncWork()
}

doRegularWork()
```

If your asynchronous work needs to be waited for, you don't have much of a choice but to mark your current code as also being `async` so that you can use `await` as normal.

However, sometimes this can result in a bit of an "async infection" – you mark one function as being async, which means its caller needs to be async too, as does *its* caller, and so on, until you've turned one error into 50.

In this situation, you can create a dedicated `Task` to solve the problem. We'll be covering this API in more detail later on, but here's how it would look in your code:

```
func doAsyncWork() async {
    print("Doing async work")
}

func doRegularWork() {
    Task {
        await doAsyncWork()
    }
}

doRegularWork()
```

Download this as an Xcode project

Tasks like this one are created and run immediately. We aren't waiting for the task to complete, so we shouldn't use `await` when creating it.

23. Whats the difference between sequence asyncsequence and asyncstream

Swift provides several ways of receiving a potentially endless flow of data, allowing us to read values one by one, or loop over them using `for`, `while`, or similar.

The simplest is the `Sequence` protocol, which continually returns values until the sequence is terminated by returning `nil`. Lots of things conform to `Sequence`, including arrays, strings, ranges, `Data`, and more. Through protocol extensions `Sequence` also gives us access to a variety of methods, including `contains()`, `filter()`, `map()`, and others.

The `AsyncSequence` protocol is almost identical to `Sequence`, with the important exception that each element in the sequence is returned asynchronously. I realize that sounds obvious, but it actually has two major impacts on the way they work.

First, reading a value from the async sequence must use `await` so the sequence can suspend itself while reading its next value. This might be performing some complex work, for example, or perhaps fetching data from a server.

Second, more advanced async sequences known as *async streams* might generate values faster than you can read them, in which case you can either discard the extra values or buffer them to be read later on.

So, in the first case think of it like your code wanting values faster than the async sequence can make them, whereas in the second case it's more like the async sequence generating data faster than than your code can read them.

Otherwise, `Sequence` and `AsyncSequence` have lots in common: the code to create a custom one yourself is almost the same, both can throw errors if you want, both get access to common functionality such as `map()`, `filter()`, `contains()`, and `reduce()`, and you can also use `break` or `continue` to exit loops over either of them.

24. How to loop over an asyncsequence using for await

You can loop over an `AsyncSequence` using Swift's regular loop types, `for`, `while`, and `repeat`, but whenever you read a value from the async sequence you must use either `await` or `try await` depending on whether it can throw errors or not.

As an example, `URL` has a built-in `lines` property that generates an async sequence of all the lines directly from a URL. This does a *lot* of work internally: making the network request, fetching part of the data, converting it into a string, sending it back for us to use, then repeating fetch, convert, and send again and again until the server stops sending back data.

All that functionality boils down to just a handful of lines of code:

```
func fetchUsers() async throws {
    let url = URL(string: "https://hws.dev/users.csv")!

    for try await line in url.lines {
        print("Received user: \(line)")
    }
}

try? await fetchUsers()
```

Download this as an Xcode project

Notice how we must use `try` along with `await`, because fetching data from the network might throw errors.

Using `lines` returns a specialized type called `AsyncLineSequence`, which returns individual lines from the download as strings. Because our source happens to be a comma-separated values file (CSV), we can write code to read the values from each line easily enough:

```
func printUsers() async throws {
    let url = URL(string: "https://hws.dev/users.csv")!

    for try await line in url.lines {
        let parts = line.split(separator: ",")
        guard parts.count == 4 else { continue }

        guard let id = Int(parts[0]) else { continue }
        let firstName = parts[1]
        let lastName = parts[2]
        let country = parts[3]

        print("Found user #\(id): \(firstName) \(lastName) from \(country)")
    }
}

try? await printUsers()
```

Download this as an Xcode project

Just like a regular sequence, using an async sequence in this way effectively generates an iterator then calls `next()` on it repeatedly until it returns `nil`, at which point the loop finishes.

If you want more control over how the sequence is read, you can of course create your own iterator then call `next()` whenever you want and as often as you want. Again, it will send back `nil` when the sequence is empty, at which point you should stop calling it.

For example, we could read the first user from our CSV and treat them specially, then read the next four users and do something specific for them, then finally reduce all the remainder down into an array of other users:

```
func printUsers() async throws {
    let url = URL(string: "https://hws.dev/users.csv")!

    var iterator = url.lines.makeAsyncIterator()

    if let line = try await iterator.next() {
        print("The first user is \(line)")
    }
```

```
    for i in 2...5 {
        if let line = try await iterator.next() {
            print("User #\(i): \(line)")
        }
    }

    var remainingResults = [String]()

    while let result = try await iterator.next() {
        remainingResults.append(result)
    }

    print("There were \(remainingResults.count) other users.")
}

try? await printUsers()
```

Download this as an Xcode project

25. How to manipulate an asyncsequence using map filter and more

`AsyncSequence` has implementations of many of the same methods that come with `Sequence`, but *how* they operate varies: some return a single value that fulfills your request, such as requesting the first value from the async sequence, and others return a new kind of async sequence, such as filtering values as they arrive.

This distinction in turn affects how they are called: returning a single value requires you to await at the call site, whereas returning a new async sequence requires you to await later on when you're reading values from the new sequence.

To demonstrate this difference, let's try out a few common operations, starting with a call to `map()`. Mapping an async sequence creates a new async sequence with the type `AsyncMapSequence`, which stores both your original async sequence and also the transformation function you want to use.

So, instead of waiting for all elements to be returned and transforming them at once, you've effectively put the transformation into a chain of work: rather than fetching an item and sending it back, the sequence now fetches an item, transforms it, *then* sends it back.

So, we could map over the lines from a `URL` to make each line uppercase, like this:

```
func shoutQuotes() async throws {
    let url = URL(string: "https://hws.dev/quotes.txt")!
    let uppercaseLines = url.lines.map(\.localizedUppercase)

    for try await line in uppercaseLines {
        print(line)
    }
}

try? await shoutQuotes()
```

Download this as an Xcode project

This also works for converting between types using `map()`, like this:

```
struct Quote {
    let text: String
}

func printQuotes() async throws {
    let url = URL(string: "https://hws.dev/quotes.txt")!

    let quotes = url.lines.map(Quote.init)

    for try await quote in quotes {
        print(quote.text)
    }
}

try? await printQuotes()
```

Download this as an Xcode project

Alternatively, we could use `filter()` to check every line with a predicate, and process only those that pass. Using our quotes, we could print only anonymous quotes like this:

```
func printAnonymousQuotes() async throws {
    let url = URL(string: "https://hws.dev/quotes.txt")!
    let anonymousQuotes = url.lines.filter { $0.contains("Anonymous") }

    for try await line in anonymousQuotes {
        print(line)
    }
}
```

```
try? await printAnonymousQuotes()
```

Download this as an Xcode project

Or we could use `prefix()` to read just the first five values from an async sequence:

```
func printTopQuotes() async throws {
    let url = URL(string: "https://hws.dev/quotes.txt")!
    let topQuotes = url.lines.prefix(5)

    for try await line in topQuotes {
        print(line)
    }
}

try? await printTopQuotes()
```

Download this as an Xcode project

And of course you can also combine these together in varying ways depending on what result you want. For example, this will filter for anonymous quotes, pick out the first five, then make them uppercase:

```
func printQuotes() async throws {
    let url = URL(string: "https://hws.dev/quotes.txt")!

    let anonymousQuotes = url.lines.filter { $0.contains("Anonymous") }
    let topAnonymousQuotes = anonymousQuotes.prefix(5)
    let shoutingTopAnonymousQuotes = topAnonymousQuotes.map(\.localizedUppercase)

    for try await line in shoutingTopAnonymousQuotes {
        print(line)
    }
}

try? await printQuotes()
```

Download this as an Xcode project

Just like using a regular `Sequence`, the order you apply these transformations matters – putting `prefix()` before `filter()` will pick out the first five quotes *then* select only the ones that are anonymous, which might produce fewer results.

Each of these transformation methods returns a new type specific to what the method does, so calling `map()` returns an `AsyncMapSequence`, calling `filter()` returns an `AsyncFilterSequence`, and calling `prefix()` returns an `AsyncPrefixSequence`.

When you stack multiple transformations together – for example, a filter, then a prefix, then a map, as in our previous example – this will inevitably produce a fairly complex return type, so if you intend to send back one of the complex async sequences you should consider an opaque return type such as `some AsyncSequence`.

All the transformations so far have created new async sequences and so we haven't needed to use them with `await`, but many also produce a single value. These *must* use `await` in order to suspend until all parts of the sequence have been returned, and may also need to use `try` if the sequence is throwing.

For example, `allSatisfy()` will check whether all elements in an async sequence pass a predicate of your choosing:

```
func checkQuotes() async throws {
    let url = URL(string: "https://hws.dev/quotes.txt")!
    let noShortQuotes = try await url.lines.allSatisfy { $0.count > 30 }
    print(noShortQuotes)
}

try? await checkQuotes()
```

Download this as an Xcode project

Important: As with regular sequences, in order to return a correct value `allSatisfy()` must have fetched every value in the sequence first, and therefore using it with an infinite sequence will never return a value. The same is true of other

similar functions, such as `min()`, `max()`, and `reduce()`, so be careful.

You can of course combine methods that create new async sequences and return a single value, for example to fetch lots of random numbers, convert them to integers, then find the largest:

```
func printHighestNumber() async throws {
    let url = URL(string: "https://hws.dev/random-numbers.txt")!

    if let highest = try await url.lines.compactMap(Int.init).max() {
        print("Highest number: \(highest)")
    } else {
        print("No number was the highest.")
    }
}

try? await printHighestNumber()
```

Download this as an Xcode project

Or to sum all the numbers:

```
func sumRandomNumbers() async throws {
    let url = URL(string: "https://hws.dev/random-numbers.txt")!
    let sum = try await url.lines.compactMap(Int.init).reduce(0, +)
    print("Sum of numbers: \(sum)")
}

try? await sumRandomNumbers()
```

Download this as an Xcode project

26. How to create a custom asyncsequence

There are only three differences between creating an `AsyncSequence` and creating a regular `Sequence`, none of which are complicated.

The differences are:

That last point technically allows us to create one type that is both a synchronous and asynchronous sequence, although I'm not sure when that would be a good idea.

We're going to build two async sequences so you can see how they work, one simple and one more realistic. First, the simple one, which is an async sequence that doubles numbers every time `next()` is called:

```
struct DoubleGenerator: AsyncSequence, AsyncIteratorProtocol {
    var current = 1

    mutating func next() async -> Int? {
        defer { current &*= 2 }

        if current < 0 {
            return nil
        } else {
            return current
        }
    }

    func makeAsyncIterator() -> DoubleGenerator {
        self
    }
}

let sequence = DoubleGenerator()

for await number in sequence {
    print(number)
}
```

Download this as an Xcode project

Tip: In case you haven't seen it before, `&*=` multiplies with overflow, meaning that rather than running out of room when the value goes beyond the highest number of a 64-bit integer, it will instead flip around to be negative. We use this to our advantage, returning `nil` when we reach that point.

If you prefer having a separate iterator struct, that also works as with `Sequence` and you don't need to adjust the calling code:

```
struct DoubleGenerator: AsyncSequence {
    struct AsyncIterator: AsyncIteratorProtocol {
        var current = 1

        mutating func next() async -> Int? {
            defer { current &*= 2 }

            if current < 0 {
                return nil
            } else {
                return current
            }
        }
    }

    func makeAsyncIterator() -> AsyncIterator {
        AsyncIterator()
    }
}

let sequence = DoubleGenerator()

for await number in sequence {
    print(number)
}
```

Download this as an Xcode project

Now let's look at a more complex example, which will periodically fetch a URL that's either local or remote, and send back any values that have changed from the previous request.

This is more complex for various reasons:

Like I said, this is more complex, but it's also a useful, real-world example of `AsyncSequence`. It's also particularly powerful when combined with SwiftUI's `task()` modifier, because the network fetches will automatically start when a view is shown and cancelled when it disappears. This allows you to constantly watch for new data coming in, and stream it directly into your UI.

Anyway, here's the code – it creates a `URLWatcher` struct that conforms to the `AsyncSequence` protocol, along with an example of it being used to display a list of users in a SwiftUI view:

```
struct URLWatcher: AsyncSequence, AsyncIteratorProtocol {
    let url: URL
    let delay: Int
    private var comparisonData: Data?
    private var isActive = true

    init(url: URL, delay: Int = 10) {
        self.url = url
        self.delay = delay
    }

    mutating func next() async throws -> Data? {
        // Once we're inactive always return nil immediately
        guard isActive else { return nil }

        if comparisonData == nil {
            // If this is our first iteration, return the initial value
            comparisonData = try await fetchData()
        } else {
            // Otherwise, sleep for a while and see if our data changed
            while true {
                try await Task.sleep(for: .seconds(delay))
                let latestData = try await fetchData()

                if latestData != comparisonData {
                    // New data is different from previous data,
                    // so update previous data and send it back
                    comparisonData = latestData
                    break
                }
            }
        }

        if comparisonData == nil {
            isActive = false
            return nil
        } else {
            return comparisonData
        }
    }

    private func fetchData() async throws -> Data {
        let (data, _) = try await URLSession.shared.data(from: url)
        return data
    }

    func makeAsyncIterator() -> URLWatcher {
        self
    }
}

// As an example of URLWatcher in action, try something like this:
struct User: Identifiable, Decodable {
    let id: Int
    let name: String
}

struct ContentView: View {
    @State private var users = [User]()

    var body: some View {
        List(users) { user in
            Text(user.name)
        }
        .task {
            // continuously check the URL watcher for data
            await fetchUsers()
        }
    }

    func fetchUsers() async {
```

```
        let url = URL(fileURLWithPath: "FILENAMEHERE.json")
        let urlWatcher = URLWatcher(url: url, delay: 3)

        do {
            for try await data in urlWatcher {
                try withAnimation {
                    users = try JSONDecoder().decode([User].self, from: data)
                }
            }
        } catch {
            // just bail out
        }
    }
}
```

Download this as an Xcode project

To make that work in your own project, replace “FILENAMEHERE” with the location of a local file you can test with.

Important: If you're trying this out with a local file on macOS, go to the Signing & Capabilities tab for your target then remove the App Sandbox capability so you can read files anywhere.

For example, I might use **/Users/twostraws/Desktop/users.json**, giving that file the following example contents:

```
[
    {
        "id": 1,
        "name": "Paul"
    }
]
```

When the code first runs the list will show Paul, but if you edit the JSON file and re-save with extra users, they will just slide into the SwiftUI list automatically.

27. How to convert an asyncsequence into a sequence

Swift does not provide a built-in way of converting an `AsyncSequence` into a regular `Sequence`, but often you'll want to make this conversion yourself so you don't need to keep awaiting results to come back in the future.

The easiest thing to do is call `reduce(into:)` on the sequence, appending each item to an array of the sequence's element type. To make this more reusable, I'd recommend adding an extension such as this one:

```
extension AsyncSequence {
    func collect() async throws -> [Element] {
        try await reduce(into: [Element]()) { $0.append($1) }
    }
}
```

With that in place, you can now call `collect()` on any async sequence in order to get a simple array of its values. Because this is an async operation, you must call it using `await` like so:

```
extension AsyncSequence {
    func collect() async throws -> [Element] {
        try await reduce(into: [Element]()) { $0.append($1) }
    }
}

func getNumberArray() async throws -> [Int] {
    let url = URL(string: "https://hws.dev/random-numbers.txt")!
    let numbers = url.lines.compactMap(Int.init)
    return try await numbers.collect()
}

if let numbers = try? await getNumberArray() {
    for number in numbers {
        print(number)
    }
}
```

Download this as an Xcode project

Tip: If you prefer, you can make `collect()` use `rethrows` rather than `throws`, so that you only need to call it using `try` if the call to `reduce()` would normally throw. This means if you have an async sequence that doesn't throw errors, you can skip `try` entirely.

28. How to create and use asyncstreams to return buffered data

`AsyncStream` and `AsyncThrowingStream` can be thought of a bit like continuations that can send back multiple values, but also like an `AsyncSequence` that is able to buffer only a certain number of values according to a policy.

Let's start with a simple stream that sends back nine integers:

```
let stream = AsyncStream { continuation in
    for i in 1...9 {
        continuation.yield(i)
    }

    continuation.finish()
}
```

We could then read and print the values from that stream using a `for await` loop, like this:

```
let stream = AsyncStream { continuation in
    for i in 1...9 {
        continuation.yield(i)
    }

    continuation.finish()
}

for await item in stream {
    print(item)
}
```

Download this as an Xcode project

Tip: Any values yielded after calling `finish()` on your continuation will be ignored.

You can see immediately that `AsyncStream` looks like using `withCheckedContinuation()`: we give it a closure to run to generate its values, and we're passed a continuation inside there that allows us to send values back to the call site.

This time, though, we can call `yield()` as many times as we want, whereas calling `continuation.resume(returning:)` could be called only once.

Because `AsyncStream` lets us send back multiple values, the only way for Swift to know the stream has ended is by us calling `finish()` on the continuation. Without that, the stream would never end, and our `for await` loop would never end – the program would just run endlessly, waiting for more data from the stream.

Tip: I would recommend trying this yourself – just comment out `continuation.finish()` and run the program again, so you can start to get a better feel for how `AsyncStream` works.

So far, this looks just like a continuation that can send back multiple values. But here's where `AsyncStream` gets clever:

Each of those are extremely powerful, so let's tackle them one by one.

First, the ability to asynchronously read values from the stream means we can read values whenever we want and as often as we want.

For example, we could do some work, then read some values from our stream, do some other work, read some more values, do still other work, and so on, like this:

```
let stream = AsyncStream { continuation in
    for i in 1...9 {
        continuation.yield(i)
    }

    continuation.finish()
}
```

```
for _ in 1...3 {
    print("The next three are:")

    for await item in stream.prefix(3) {
        print(item)
    }
}
```

Download this as an Xcode project

Warning: Calling `prefix()` on an `AsyncStream` returns a dedicated type called `AsyncPrefixSequence`, which will read *N* values from the stream each time you loop over it rather than acting as a slice.

So, this code will print the same output as accessing `stream.prefix(3)` directly each time:

```
let stream = AsyncStream { continuation in
    for i in 1...9 {
        continuation.yield(i)
    }

    continuation.finish()
}

let firstThree = stream.prefix(3)

for _ in 1...3 {
    print("The next three are:")

    for await item in firstThree {
        print(item)
    }
}
```

Download this as an Xcode project

The second power feature of `AsyncStream` is how smoothly it lets us read values concurrently. So, we can read values from our stream in three separate tasks, and each will get some of the values from the stream:

```
let stream = AsyncStream { continuation in
    for i in 1...9 {
        continuation.yield(i)
    }

    continuation.finish()
}

Task {
    for await item in stream {
        print("1. \(item)")
    }
}

Task {
    for await item in stream {
        print("2. \(item)")
    }
}

Task {
    for await item in stream {
        print("3. \(item)")
    }
}

try? await Task.sleep(for: .seconds(1))
```

Download this as an Xcode project

When that runs, you'll probably see each task print out three values, interleaved with output from other tasks.

Important: Accessing an `AsyncStream` concurrently means each task will receive only some of the values from the stream. If you need every task to receive every value, you should look at a Combine publisher instead.

Finally, the ability to add values into the stream means we can place values into our stream whenever we want, take a break for a while, add some more, and so on.

In our current code, all nine integers get placed into the stream before anything is read – we don't know when or where they will be read, or even if they will be read at all. But we could equally well add numbers over time, like this:

```
let stream = AsyncStream { continuation in
    for i in 1...3 {
        Task {
            try await Task.sleep(for: .seconds(i))
            continuation.yield(i)

            if i == 3 {
                continuation.finish()
            }
        }
    }
}

for await item in stream {
    print(item)
}
```

Download this as an Xcode project

All three of those `AsyncStream` features are useful, but it has one other power that takes it up into a league of its own: we can set a buffering policy that determines how values get added when existing values have yet to be read.

There are three buffering policies to choose from:

For example, we could use `.bufferingNewest(5)` to allow our stream to receive five values as normal, but if it receives a sixth one before the first has been read it will discard the first value.

You can see this in action here:

```
let stream = AsyncStream(bufferingPolicy: .bufferingNewest(5)) { continuation in
    for i in 1...9 {
        continuation.yield(i)
    }

    continuation.finish()
}

for await item in stream {
    print(item)
}
```

Download this as an Xcode project

It's worth spending a moment to really think what's happening there:

- Swift adds 1, 2, 3, 4, and 5 to the stream as normal
- When 6 comes in the buffer is full. So, it discards 1 (the oldest value) to make room, and adds 6 at the end.
- This then happens again for 7, this time discarding the next oldest value, which is 2.
- 8 then comes in, so 3 is discarded.
- Finally, 9 is added to the stream, so 4 is discarded.
- All this happens before our `for await` loop runs, which means by the time it *does* run the stream contains 5, 6, 7, 8, 9, which gets printed.

If you use a buffer size of 1, it means one of two things:

- Using `bufferingOldest(1)` means "don't add any new values to the stream until the first one was read."
- Using `bufferingNewest(1)` means "always throw away the existing value when a new one comes in."

Using `bufferingNewest(1)` allows your stream to store the latest value ready for reading as soon as anyone asks for it. They'll also then get all future values for as long as they continue to read from the stream.

If you use a buffer size of zero, the value *must* be read straight away otherwise it will be discarded. **To be clear: if nothing is actively reading from the stream, any values added with a zero buffer will be discarded.**

You can see the problem in the following code:

```
let stream = AsyncStream(bufferingPolicy: .bufferingOldest(0)) { continuation in
    continuation.yield("Hello, AsyncStream!")
    continuation.finish()
}

for await item in stream {
    print(item)
}
```

Download this as an Xcode project

That posts creates a stream and posts a value before anything attempts to read from the stream, so it gets discarded immediately.

In comparison, this next code only yields its value after one second has elapsed:

```
let stream = AsyncStream(bufferingPolicy: .bufferingOldest(0)) { continuation in
    Task {
        try await Task.sleep(for: .seconds(1))
        continuation.yield("Hello, AsyncStream!")
        continuation.finish()
    }
}

for await item in stream {
    print(item)
}
```

Download this as an Xcode project

So, by the time a value is yielded, our `for await` loop is actively waiting for it.

Zero-length buffers are helpful for situations when a value makes sense only when consumed right now. For example, if you wanted to watch the filesystem for changes, you wouldn't want to be notified of all changes that happened before you started watching, only those that happen *after*.

Finally, `AsyncThrowingStream` is useful for places when the work you will do might finish early by throwing an error. **This will end the stream immediately** – all previous values will be sent back as normal, but no future values will.

For example, we could make our number stream throw an error when it's asked to return anything that's a multiple of 3:

```
enum MultipleError: Error {
    case no3
}

let stream = AsyncThrowingStream { continuation in
    for i in 1...40 {
        continuation.yield(i)

        if i.isMultiple(of: 3) {
            continuation.finish(throwing: MultipleError.no3)
        }
    }

    continuation.finish()
}
```

As you can see, throwing an error is done by calling `continuation.finish(throwing:)` – it's really clear that this finishes the stream. In this code we could actually add a `return` keyword after throwing the error, but in your own code you might want to add some clean up work.

We can now loop over the values from that stream, handling errors somehow:

```
enum MultipleError: Error {
    case no3
}

let stream = AsyncThrowingStream { continuation in
```

```
    for i in 1...40 {
        continuation.yield(i)

        if i.isMultiple(of: 3) {
            continuation.finish(throwing: MultipleError.no3)
        }
    }

    continuation.finish()
}

do {
    for try await values in stream {
        print(values)
    }
} catch {
    print("Error received: \(error)")
}
```

Download this as an Xcode project

When that code runs, it will print 1, 2, and 3, then the error is thrown. 4 and 5 won't be printed, because the stream ends before they are yielded.

29. What are tasks and task groups

Using async/await in Swift allows us to write asynchronous code that is easy to read and understand, but by itself it doesn't enable us to run anything concurrently – even with several CPU cores working hard, async/await code would still execute sequentially.

To create actual concurrency – to provide the ability for multiple pieces of work to run at the same time – Swift provides us with two specific types for constructing and managing concurrency in a way that makes it easier to use: `Task` and `TaskGroup`. Although the types themselves aren't *complex*, they unlock a lot of power and flexibility, and sit at the core of how we use concurrency with Swift.

Which you choose – `Task` or `TaskGroup` – depends on the goal of your work: if you want one or two independent pieces of work to start, then `Task` is the right choice. If you want to split up one job into several concurrent operations then `TaskGroup` is a better fit. Task groups work best when their individual operations return exactly the same kind of data, but with a little extra effort you can coerce them into supporting heterogenous data types.

Although you might not realize it, you're using tasks every time you write any async code in Swift. You see, all async functions run as part of a task whether or not we explicitly ask for it to happen. Even using `async let` is *syntactic sugar* for creating a task then waiting for its result – special syntax that makes a particular piece of code easier to write. This is why if you use multiple sequential `async let` calls they will all start executing immediately while the rest of your code continues.

Both `Task` and `TaskGroup` can be created with one of four priority levels: `high` is the most important, then `medium`, `low`, and finally `background` at the bottom. Task priorities allow the system to adjust the order in which it executes work, meaning that important work can happen before unimportant work.

Tip: If you've been doing iOS programming for a while, you may prefer to use the more familiar quality of service priorities from `DispatchQueue`, which are `userInitiated` and `utility` in place of `high` and `low` respectively. There is no equivalent to the old `userInteractive` priority, which is now exclusively reserved for the user interface.

30. How to create and run a task

Swift's `Task` struct lets us start running some work immediately, and optionally wait for the result to be returned. And it *is* optional: sometimes you don't care about the result of the task, or sometimes the task automatically updates some external value when it completes, so you can just use them as "fire and forget" operations if you need to. This makes them a great way to run async code from a synchronous function.

First, let's look at an example where we create two tasks back to back, then wait for them both to complete. This will fetch data from two different URLs, decode them into two different structs, then print a summary of the results, all to simulate a user starting up a game – what are the latest news updates, and what are the current highest scores?

Here's how that looks:

```
struct NewsItem: Decodable {
    let id: Int
    let title: String
    let url: URL
}

struct HighScore: Decodable {
    let name: String
    let score: Int
}

func fetchUpdates() async {
    let newsTask = Task {
        let url = URL(string: "https://hws.dev/headlines.json")!
        let (data, _) = try await URLSession.shared.data(from: url)
        return try JSONDecoder().decode([NewsItem].self, from: data)
    }

    let highScoreTask = Task {
        let url = URL(string: "https://hws.dev/scores.json")!
        let (data, _) = try await URLSession.shared.data(from: url)
        return try JSONDecoder().decode([HighScore].self, from: data)
    }

    do {
        let news = try await newsTask.value
        let highScores = try await highScoreTask.value
        print("Latest news loaded with \(news.count) items.")

        if let topScore = highScores.first {
            print("\(topScore.name) has the highest score with \(topScore.score), out of \(highScores.count) total
        }
    } catch {
        print("There was an error loading user data.")
    }
}

await fetchUpdates()
```

Download this as an Xcode project

Let's unpick the key parts:

Both tasks in that example returned a value, but that's not a requirement – the "fire and forget" approach allows us to create a task without storing it, and Swift will ensure it runs until completion correctly.

To demonstrate this, we could make a small SwiftUI program to fetch a user's inbox when a button is pressed. Button actions are *not* async functions, so we need to launch a new task inside the action. The task *can* call async functions, but in this instance we don't actually care about the result so we're not going to store the task – the function it calls will handle updating our SwiftUI view.

Here's the code:

```
struct Message: Decodable, Identifiable {
    let id: Int
    var from: String
    var text: String
}

struct ContentView: View {
```

```
    @State private var messages = [Message]()

    var body: some View {
        NavigationStack {
            Group {
                if messages.isEmpty {
                    Button("Load Messages") {
                        Task {
                            await loadMessages()
                        }
                    }
                } else {
                    List(messages) { message in
                        VStack(alignment: .leading) {
                            Text(message.from)
                                .font(.headline)

                            Text(message.text)
                        }
                    }
                }
            }
            .navigationTitle("Inbox")
        }
    }

    func loadMessages() async {
        do {
            let url = URL(string: "https://hws.dev/messages.json")!
            let (data, _) = try await URLSession.shared.data(from: url)
            messages = try JSONDecoder().decode([Message].self, from: data)
        } catch {
            messages = [
                Message(id: 0, from: "Failed to load inbox.", text: "Please try again later.")
            ]
        }
    }
}
```

Download this as an Xcode project

Even though that code isn't so different from the previous example, I still want to pick out a few things:

I know it's a not a lot of code, but between `Task`, async/await, and SwiftUI a lot of work is happening on our behalf. Remember, when we use `await` we're signaling a potential suspension point, which means the task might sleep for a while based on the work that's happening.

Let's break it down:

- All UI work runs on the main thread, so the button's action closure will fire on the main thread.
- We create the task on the main thread, and the code we're running belongs to our SwiftUI view, so it will also run on the main thread.
- Inside `loadMessages()` we use `await` to load our URL data, but that will run on its own networking thread to avoid making our UI freeze. When it resumes, our code will return to the main thread.
- Finally, the `messages` property uses the `@State` property wrapper, which will automatically update its value on the main thread no matter where we change it from.

Best of all, we don't have to care about this – we don't need to know how the system is balancing the threads, or even that the threads exist, because Swift and SwiftUI take care of that for us. In fact, the concept of tasks is so thoroughly baked into SwiftUI that there's a dedicated `task()` modifier that makes them even easier to use.

31. Whats the difference between a task and a detached task

If you create a new task using the regular `Task` initializer, your work starts running immediately and inherits the priority of the caller, any task local values, and its actor context. On the other hand, detached tasks also start work immediately, but do *not* inherit the priority or other information from the caller.

I'm going to explain in more detail why these differences matter, but first I want to mention this very important quote from the Swift Evolution proposal for `async let`: "`Task.detached` most of the time should not be used at all." I'm getting that out of the way up front so you don't spend time learning about detached tasks, only to realize you probably shouldn't use them!

Still here? Okay, let's dig in to our three differences: priority, task local values, and actor isolation.

The priority part is straightforward: if you're inside a high-priority task and create a new task, it will also have a high priority, whereas creating a new detached task would have a nil priority unless you specifically asked for something.

The task local values part is a little more complex, but to be honest probably isn't going to be of interest to most people. Task local values allow us to share a specific value everywhere inside one specific task – they are like static properties on a type, except rather than everything sharing that property, each task has its own value. Detached tasks do *not* inherit the task local values of their parent because they do not have a parent.

The *actor context* part is more important and more complex. When you create a regular task from inside an actor it will be isolated to that actor, which means you can use other parts of the actor synchronously:

```
actor User {
    func authenticate(user: String, password: String) -> Bool {
        // Complicated logic here
        return true
    }

    func login() {
        Task {
            if authenticate(user: "taytay89", password: "n3wy0rk") {
                print("Successfully logged in.")
            } else {
                print("Sorry, something went wrong.")
            }
        }
    }
}

let user = User()
await user.login()

try? await Task.sleep(for: .seconds(0.5))
```

Download this as an Xcode project

In comparison, a detached task runs concurrently with all other code, including the actor that created it – it effectively has no parent, and therefore has greatly restricted access to the data inside the actor.

So, if we were to rewrite the previous actor to use a detached task, it would need to call `authenticate()` like this:

```
actor User {
    func login() {
        Task.detached {
            if await self.authenticate(user: "taytay89", password: "n3wy0rk") {
                print("Successfully logged in.")
            } else {
                print("Sorry, something went wrong.")
            }
        }
    }

    func authenticate(user: String, password: String) -> Bool {
        // Complicated logic here
        return true
    }
```

```
}

let user = User()
await user.login()
```

Download this as an Xcode project

This distinction is particularly important when you are running on the main actor, which will be the case if you're responding to a button click for example. The rules here might not be immediately obvious, so I want to show you some examples of what is allowed and what is *not* allowed, and more importantly explain why each is the case.

First, if you're changing the value of an `@State` property, you can do so using a regular task like this:

```
struct ContentView: View {
    @State private var name = "Anonymous"

    var body: some View {
        VStack {
            Text("Hello, \(name)!")
            Button("Authenticate") {
                Task {
                    name = "Taylor"
                }
            }
        }
    }
}
```

Download this as an Xcode project

Note: The `Task` here is of course not needed because we're just setting a local value; I'm just trying to illustrate how regular tasks and detached tasks are different.

If we try and do the same thing from a *detached* task, Swift will complain that it's not safe, even though one would hope that `@State` should be able to synchronize with the main actor just fine.

The rules are similar when we switch to an object that uses the `@Observable` macro to publish changes: because the entire SwiftUI view runs on the main actor, any changes made to the `@Observable` object from there will also run on the main actor.

So, this kind of code is safe:

```
@Observable
class ViewModel {
    var name = "Anonymous"
}

struct ContentView: View {
    @State private var model = ViewModel()

    var body: some View {
        VStack {
            Text("Hello, \(model.name)!")
            Button("Authenticate") {
                Task {
                    model.name = "Taylor"
                }
            }
        }
    }
}
```

Download this as an Xcode project

However, we *cannot* use `Task.detached` here, because we're passing a property made on the main actor into a task running on a different actor entirely.

At this point, you might wonder why detached tasks would have any use. Well, consider this code:

```
struct ContentView: View {
    var body: some View {
```

```
            Button("Authenticate", action: doWork)
        }

        func doWork() {
            Task {
                for i in 1...10_000 {
                    print("In Task 1: \(i)")
                }
            }

            Task {
                for i in 1...10_000 {
                    print("In Task 2: \(i)")
                }
            }
        }
    }
```

Download this as an Xcode project

That's the simplest piece of code that demonstrates the usefulness of detached tasks: a SwiftUI view with a button that launches a couple of tasks to print out text.

When that runs, you'll see "In Task 1" printed 10,000 times, then "In Task 2" printed 10,000 times – even though we have created two tasks, they are executing sequentially. This happens because our SwiftUI views always run on the main actor – it's literally baked into the `View` protocol – and so only one task can run at a time.

In contrast, if you change both `Task` initializers to `Task.detached`, you'll see "In Task 1" and "In Task 2" get intermingled as both execute at the same time. Without any need for actor isolation, Swift can run those tasks in parallel – using a detached task has allowed us to shed our attachment to the main actor.

Although detached tasks do have very specific uses, generally I think they should be your last port of call – use them only if you've tried both a regular task and `async let`, and neither solved your problem.

32. How to make a task sleep

Swift's `Task` struct has a static `sleep()` method that will cause the current task to be suspended for a set period of time. You need to call `Task.sleep()` using `await` as it will cause the task to be suspended, and you also need to use `try` because `sleep()` will wake and throw an error if the task is cancelled.

For example, this will make the current task sleep for at least 3 seconds:

```
try await Task.sleep(for: .seconds(3))
```

Calling `Task.sleep()` will make the current task sleep for *at least* the amount of time you ask, not *exactly* the time you ask. There is a little drift involved because the system might be busy doing other work when the sleep ends, but you are at least guaranteed it won't end *before* your time has elapsed.

If you're happy to add a little extra drift, you can add a `tolerance` parameter that allows the task to sleep for longer if needed. So, this might sleep for three seconds up to a total of four seconds:

```
try await Task.sleep(for: .seconds(3), tolerance: .seconds(1))
```

Swift's `Task.sleep()` method automatically checks for cancellation, meaning that if you cancel a sleeping task it will be woken and throw a `CancellationError` for you to catch.

Tip: Unlike making a thread sleep, `Task.sleep()` does *not* block the underlying thread, allowing it pick up work from elsewhere if needed.

33. How to get a result from a task

If you want to read the return value from a `Task` directly, you should read its `value` using `await`, or use `try await` if it has a throwing operation. However, all tasks also have a `result` property that returns an instance of Swift's `Result` struct, generic over the type returned by the task as well as whether it might contain an error or not.

To demonstrate this, we could write some code that creates a task to fetch and decode a string from a URL. To start with we're going to make this task throw errors if the download fails, or if the data can't be converted to a string.

Here's the code:

```
enum LoadError: Error {
    case fetchFailed, decodeFailed
}

func fetchQuotes() async {
    let downloadTask = Task {
        let url = URL(string: "https://hws.dev/quotes.txt")!

        do {
            let (data, _) = try await URLSession.shared.data(from: url)

            if let string = String(data: data, encoding: .utf8) {
                return string
            } else {
                throw LoadError.decodeFailed
            }
        } catch {
            throw LoadError.fetchFailed
        }
    }

    let result = await downloadTask.result

    do {
        let string = try result.get()
        print(string)
    } catch LoadError.fetchFailed {
        print("Unable to fetch the quotes.")
    } catch LoadError.decodeFailed {
        print("Unable to convert quotes to text.")
    } catch {
        print("Unknown error.")
    }
}

await fetchQuotes()
```

Download this as an Xcode project

There's not a lot of code there, but there are a few things I want to point out as being important:

If you don't care what errors are thrown, or don't mind digging through Foundation's various errors yourself, you can avoid handling errors in the task and just let them propagate up:

```
func fetchQuotes() async {
    let downloadTask = Task {
        let url = URL(string: "https://hws.dev/quotes.txt")!
        let (data, _) = try await URLSession.shared.data(from: url)
        return String(decoding: data, as: UTF8.self)
    }

    let result = await downloadTask.result

    do {
        let string = try result.get()
        print(string)
    } catch {
        print("Unknown error.")
    }
}

await fetchQuotes()
```

Download this as an Xcode project

The main take aways here are:

Many places where `Result` was useful are now better served through async/await, but `Result` is still useful for storing in a single value the success or failure of some operation. Yes, in the code above we evaluate the result immediately for brevity, but the power of `Result` is that it's a value you can pass around elsewhere in your code to deal with at a later time.

34. How to control the priority of a task

Swift tasks can have a priority attached to them, such as `.high` or `.background`, but the priority can also be `nil` if no specific priority was assigned. This priority can be used by the system to determine which task should be executed next, but this isn't guaranteed – think of it as a suggestion rather than a rule.

Creating a task with a priority looks like this:

```
func fetchData() async {
    let downloadTask = Task(priority: .high) {
        let url = URL(string: "https://hws.dev/chapter.txt")!
        let (data, _) = try await URLSession.shared.data(from: url)
        return String(decoding: data, as: UTF8.self)
    }

    do {
        let text = try await downloadTask.value
        print(text)
    } catch {
        print(error.localizedDescription)
    }
}

await fetchData()
```

Download this as an Xcode project

Although you *can* directly assign a priority to a task when it's created, if you don't then Swift will follow three rules for deciding the priority automatically:

This means not specifying an exact priority is often a good idea because Swift will do The Right Thing.

However, like I said you can also specify an exact priority from one of the following:

- The highest priority is `.high`, which is synonymous with the old `.userInitiated` priority from Grand Central Dispatch (GCD). As the name implies, this should be used only for tasks that the user specifically started and is actively waiting for.
- Next highest is `.medium`, which is a great choice for most of your tasks that the user isn't actively waiting for.
- Next is `.low`, which is synonymous with the old `.utility` priority from GCD. This is the best choice for anything long enough to require a progress bar to be displayed, such as copying files or importing data.
- The lowest priority is `.background`, which is for any work the user can't see, such as building a search index. This could in theory take hours to complete.

Like I said, priority inheritance helps get us a sensible priority by default, particularly when creating tasks in response to a user interface action.

For example, we could build a simple SwiftUI app using a single task, and we don't need to provide a specific priority – it will automatically run as high priority because it was started from our UI:

```
struct ContentView: View {
    @State private var jokeText = ""

    var body: some View {
        VStack {
            Text(jokeText)
            Button("Fetch new joke", action: fetchJoke)
        }
    }

    func fetchJoke() {
        Task {
            let url = URL(string: "https://icanhazdadjoke.com")!
            var request = URLRequest(url: url)
            request.setValue("Swift Concurrency by Example", forHTTPHeaderField: "User-Agent")
            request.setValue("text/plain", forHTTPHeaderField: "Accept")

            let (data, _) = try await URLSession.shared.data(for: request)

            if let jokeString = String(data: data, encoding: .utf8) {
```

```
                jokeText = jokeString
            } else {
                jokeText = "Load failed."
            }
        }
    }
}
```

Download this as an Xcode project

Any task can query its current priority using `Task.currentPriority`, but this works from anywhere – if it's called in a function that is not currently part of a task, Swift will query the system for an answer or send back `.medium`.

35. Understanding how priority escalation works

Every task can be created with a specific priority level, or it can inherit a priority from somewhere else. But in two specific circumstances, Swift will *raise* the priority of a task so it's able to complete faster.

This always happens because of some specific action from us. For example, if high-priority task A starts waiting for the result of low-priority task B, task B will have its priority elevated to the same priority as task A.

You can see this for yourself with a little code:

```
@main
struct App {
    static func main() async throws {
        // Create a high-priority task
        let outerTask = Task(priority: .high) {
            print("Outer: \(Task.currentPriority)")

            // Create a nested, low-priority task inside there.
            let innerTask = Task(priority: .low) {
                print("Inner: \(Task.currentPriority)")

                try await Task.sleep(for: .seconds(1))

                print("Inner: \(Task.currentPriority)")
            }

            // Make the high-priority task wait for its low-priority child
            try await Task.sleep(for: .seconds(0.5))
            _ = try await innerTask.value
        }

        // Wait for the whole thing to finish
        _ = try await outerTask.value
    }
}
```

Download this as an Xcode project

Here we have a high-priority task using `await` on a low-priority task, and so you'll see the inner task's priority will report `low` initially, then move up to report `high` as soon as the outer task starts waiting for it – `Task.currentPriority` will report the *escalated* priority rather than the original priority.

The other place this happens is when low-priority task A has started running on an actor, and high-priority task B has been enqueued on that same actor, task A will have its priority elevated to match task B to avoid creating a jam.

In both cases, Swift is trying to ensure the higher priority task gets the quality of service it needs to run quickly – if something very important can only complete when something less important is also complete, then the less important task *becomes* very important.

For the most part, priority escalation isn't something we need to worry about in our code – think of it as a bonus feature provided automatically by Swift's tasks.

At the time of writing, this is the only real "gotcha" moment with task escalation. The other situation – if you queue a high-priority task on the same actor where a low-priority task is already running – will also involve priority escalation, but *won't* change the value of `currentPriority`. This means your task will run a little faster and it might not be obvious why, but honestly it's unlikely you'll even notice this.

36. How to cancel a task

Swift's tasks use *cooperative cancellation*, which means that although we can tell a task to stop work, the task itself is free to completely ignore that instruction and carry on for as long as it wants. This is a feature rather than a bug: if cancelling a task made it stop work immediately, the task might leave your program in an inconsistent state.

There are seven things to know when working with task cancellation:

We can explore a few of these in code. First, here's a function that uses a task to fetch some data from a URL, decodes it into an array, then returns the average:

```
func getAverageTemperature() async {
    let fetchTask = Task {
        let url = URL(string: "https://hws.dev/readings.json")!
        let (data, _) = try await URLSession.shared.data(from: url)
        let readings = try JSONDecoder().decode([Double].self, from: data)
        let sum = readings.reduce(0, +)
        return sum / Double(readings.count)
    }

    do {
        let result = try await fetchTask.value
        print("Average temperature: \(result)")
    } catch {
        print("Failed to get data.")
    }
}

await getAverageTemperature()
```

Download this as an Xcode project

Now, there is no explicit cancellation in there, but there *is* implicit cancellation because the `URLSession.shared.data(from:)` call will check to see whether its task is still active before continuing. If the task has been cancelled, `data(from:)` will automatically throw a `URLError` and the rest of the task won't execute.

However, that implicit check happens *before* the network call, so it's unlikely to be an actual cancellation point in practice. As most of our users are likely to be using mobile network connections, the network call is likely to take most of the time of this task, particularly if the user has a poor connection.

So, we could upgrade our task to explicitly check for cancellation *after* the network request, using `Task.checkCancellation()`. This is a static function call because it will always apply to whatever task it's called inside, and it needs to be called using `try` so that it can throw a `CancellationError` if the task *has* been cancelled.

Here's the new function:

```
func getAverageTemperature() async {
    let fetchTask = Task {
        let url = URL(string: "https://hws.dev/readings.json")!
        let (data, _) = try await URLSession.shared.data(from: url)
        try Task.checkCancellation()
        let readings = try JSONDecoder().decode([Double].self, from: data)
        let sum = readings.reduce(0, +)
        return sum / Double(readings.count)
    }

    do {
        let result = try await fetchTask.value
        print("Average temperature: \(result)")
    } catch {
        print("Failed to get data.")
    }
}

await getAverageTemperature()
```

Download this as an Xcode project

As you can see, it just takes one call to `Task.checkCancellation()` to make sure our task isn't wasting time calculating data that's no longer needed.

If you want to handle cancellation yourself – if you need to clean up some resources or perform some other calculations, for example – then instead of calling `Task.checkCancellation()` you should check the value of `Task.isCancelled` instead. This is a simple Boolean that returns the current cancellation state, which you can then act on however you want.

To demonstrate this, we could rewrite our function a third time so that cancelling the task or failing to fetch data returns an average temperature of 0. This time we're going to cancel the task ourselves as soon as it's created, but because we're always returning a default value we no longer need to handle errors when reading the task's result:

```
func getAverageTemperature() async {
    let fetchTask = Task {
        let url = URL(string: "https://hws.dev/readings.json")!

        do {
            let (data, _) = try await URLSession.shared.data(from: url)
            if Task.isCancelled { return 0.0 }

            let readings = try JSONDecoder().decode([Double].self, from: data)
            let sum = readings.reduce(0, +)
            return sum / Double(readings.count)
        } catch {
            return 0.0
        }
    }

    fetchTask.cancel()

    let result = await fetchTask.value
    print("Average temperature: \(result)")
}

await getAverageTemperature()
```

Download this as an Xcode project

Tip: That uses `return 0.0` rather than `return 0` so that Swift can correctly infer the task's return type as a `Double`. If you wanted to avoid that, you could tell Swift explicitly that a `Double` is coming back using `Task { () -> Double in`.

Now we have one implicit cancellation point with the `data(from:)` call, and an explicit one with the check on `Task.isCancelled`. If either one is triggered, the task will return 0 rather than throw an error.

Tip: You can use both `Task.checkCancellation()` and `Task.isCancelled` from both synchronous and asynchronous functions. Remember, async functions can call synchronous functions freely, so checking for cancellation can be just as important to avoid doing unnecessary work.

37. How to voluntarily suspend a task

If you're executing a long-running task that has few if any suspension points, for example if you're repeatedly iterating over an intensive loop, you can call `Task.yield()` to *voluntarily* suspend the current task so that Swift can give other tasks the chance to proceed a little if needed.

To demonstrate this, we could write a simple function to calculate the factors for a number – numbers that divide another number evenly. For example, the factors for 12 are 1, 2, 3, 4, 6, and 12. A simple version of this function might look like this:

```
func factors(for number: Int) async -> [Int] {
    var result = [Int]()

    for check in 1...number {
        if number.isMultiple(of: check) {
            result.append(check)
        }
    }

    return result
}

let results = await factors(for: 120)
print("Found \(results.count) factors for 120.")
```

Download this as an Xcode project

Despite being a pretty inefficient implementation, in release builds that will still execute quite fast even for numbers such as 100,000,000. But if you try something even bigger you'll notice it struggles – running hundreds of millions of checks is really going to make the task chew up a lot of CPU time, which might mean other tasks are left sitting around unable to make even the slightest progress forward.

Keep in mind our other tasks might be able to kick off some work then suspend immediately, such as making network requests. A simple improvement is to force our `factors()` factor to pause every so often so that Swift can run other tasks if it wants – we're effectively asking it to come up for air and let another task have a go.

So, we could modify the function so that it calls `Task.yield()` every 100,000 numbers, like this:

```
func factors(for number: Int) async -> [Int] {
    var result = [Int]()

    for check in 1...number {
        if check.isMultiple(of: 100_000) {
            await Task.yield()
        }

        if number.isMultiple(of: check) {
            result.append(check)
        }
    }

    return result
}

let factors = await factors(for: 120)
print("Found \(factors.count) factors for 120.")
```

Download this as an Xcode project

However, that has the downside of now having twice as much work in the loop. As an alternative, you could try yielding only when a multiple is actually found, like this:

```
func factors(for number: Int) async -> [Int] {
    var result = [Int]()

    for check in 1...number {
        if number.isMultiple(of: check) {
            result.append(check)
            await Task.yield()
```

```
            }
        }

        return result
    }

    let factors = await factors(for: 120)
    print("Found \(factors.count) factors for 120.")
```

Download this as an Xcode project

That offers Swift the chance to pause every time a multiple is found. Yes, it will be called a lot in the first few iterations, but fewer multiples will be found over time and so it probably won't yield as often as the previous example – it could well defeat the point of using `yield()` in the first place.

Calling `yield()` does not always mean the task will stop running: if it has a higher priority than other tasks that are waiting, it's entirely possible your task will just immediately resume its work. It's *guidance* – we're giving Swift the opportunity to execute other tasks temporarily rather than forcing it to do so.

Think of calling `Task.yield()` as the equivalent of calling a fictional `Task.doNothing()` method – it gives Swift the chance to adjust the execution of its tasks without actually creating any real work.

38. How to create a task group and add tasks to it

Swift's task groups are collections of tasks that work together to produce a single result. Each task inside the group must return the same kind of data, but if you use enum associated values you can make them send back different kinds of data – it's a little clumsy, but it works.

Creating a task group is done in a very precise way to avoid us creating problems for ourselves: rather than creating a `TaskGroup` instance directly, we do so by calling the `withTaskGroup(of:)` function and telling it the data type the task group will return. We give this function the code for our group to execute, and Swift will pass in the `TaskGroup` that was created, which we can then use to add tasks to the group.

First, I want to look at the simplest possible example of task groups, which is returning 5 constant strings, adding them into a single array, then joining that array into a string:

```
func printMessage() async {
    let string = await withTaskGroup(of: String.self) { group in
        group.addTask { "Hello" }
        group.addTask { "From" }
        group.addTask { "A" }
        group.addTask { "Task" }
        group.addTask { "Group" }

        var collected = [String]()

        for await value in group {
            collected.append(value)
        }

        return collected.joined(separator: " ")
    }

    print(string)
}

await printMessage()
```

Download this as an Xcode project

I know it's trivial, but it demonstrates several important things:

However, there's one other thing you *can't* see in that code sample, which is that our task results are sent back in *completion* order and not creation order.

That is, our code above might send back "Hello From A Task Group", but it also might send back "Task From A Hello Group", "Group Task A Hello From", or any other possible variation – the return value could be different every time.

Note: From Swift 6.1 onwards, you can create `withTaskGroup()` without using the `of` parameter – Swift figures out what type to use based on the first child task you add to the group.

Tasks created using `withTaskGroup()` cannot throw errors. If you *want* them to be able to throw errors that bubble upwards – i.e., that are handled outside the task group – you should use `withThrowingTaskGroup()` instead.

To demonstrate this, and also to demonstrate a more real-world example of `TaskGroup` in action, we could write some code that fetches several news feeds and combines them into one list:

```
struct NewsStory: Decodable, Identifiable {
    let id: Int
    let title: String
    let strap: String
    let url: URL
}

struct ContentView: View {
    @State private var stories = [NewsStory]()

    var body: some View {
        NavigationStack {
            List(stories) { story in
                VStack(alignment: .leading) {
                    Text(story.title)
```

```
                        .font(.headline)

                    Text(story.strap)
                }
            }
            .navigationTitle("Latest News")
        }
        .task {
            await loadStories()
        }
    }

    func loadStories() async {
        do {
            stories = try await withThrowingTaskGroup(of: [NewsStory].self) { group in
                for i in 1...5 {
                    group.addTask {
                        let url = URL(string: "https://hws.dev/news-\(i).json")!
                        let (data, _) = try await URLSession.shared.data(from: url)
                        return try JSONDecoder().decode([NewsStory].self, from: data)
                    }
                }

                var allStories = [NewsStory]()

                for try await stories in group {
                    allStories.append(contentsOf: stories)
                }

                return allStories.sorted { $0.id > $1.id }
            }
        } catch {
            print("Failed to load stories")
        }
    }
}
```

Download this as an Xcode project

In that code you can see we have a simple struct that contains one news story, a SwiftUI view showing all the news stories we fetched, plus a `loadStories()` method that handles fetching and decoding several news feeds into a single array.

There are four things in there that deserve special attention:

Regardless of whether you're using throwing or non-throwing tasks, all tasks in a group must complete before the group returns. You have three options here:

Of the three, I find myself using the first most often because it's the most explicit – you aren't leaving folks wondering why some or all of your tasks are launched then ignored.

39. How to cancel a task group

Swift's task groups can be cancelled in one of three ways: if the parent task of the task group is cancelled, if you explicitly call `cancelAll()` on the group, or if one of your child tasks throws an uncaught error, all remaining tasks will be implicitly cancelled.

The first of those happens outside of the task group, but the other two are worth investigating.

First, calling `cancelAll()` will cancel all remaining tasks. As with standalone tasks, cancelling a task group is *cooperative*: your child tasks can check for cancellation using `Task.isCancelled` or `Task.checkCancellation()`, but they can ignore cancellation entirely if they want.

I'll show you a real-world example of `cancelAll()` in action in a moment, but before that I want to show you some toy examples so you can see how it works.

We could write a simple `printMessage()` function like this one, creating three tasks inside a group in order to generate a string:

```
func printMessage() async {
    let result = await withThrowingTaskGroup(of: String.self) { group in
        group.addTask { "Testing" }
        group.addTask { "Group" }
        group.addTask { "Cancellation" }

        group.cancelAll()
        var collected = [String]()

        do {
            for try await value in group {
                collected.append(value)
            }
        } catch {
            print(error.localizedDescription)
        }

        return collected.joined(separator: " ")
    }

    print(result)
}

await printMessage()
```

Download this as an Xcode project

As you can see, that calls `cancelAll()` immediately after creating all three tasks, and yet when the code is run you'll still see all three strings printed out.

I've said it before, but it bears repeating and this time in bold: **cancelling a task group is cooperative, so unless the tasks you add implicitly or explicitly check for cancellation calling `cancelAll()` by itself won't do much.**

To see `cancelAll()` actually working, try replacing the first `addTask()` call with this:

```
group.addTask {
    try Task.checkCancellation()
    return "Testing"
}
```

And now our behavior will be different: you might see "Cancellation" by itself, "Group" by itself, "Cancellation Group", "Group Cancellation", or nothing at all.

To understand why, keep the following in mind:

When you put those together, it's entirely possible the first task to complete is the one that calls `Task.checkCancellation()`, which means our loop will exit, we'll print an error message, and send back an empty string. Alternatively, one or both of the other tasks might run first, in which case we'll get our other possible outputs.

Remember, calling `cancelAll()` only cancels *remaining* tasks, meaning that it won't undo work that has already completed. Even then the cancellation is cooperative, so you need to make sure the tasks you add to the group check for cancellation.

With that toy example out of the way, here's a more complex demonstration of `cancelAll()` that builds on an example from an earlier chapter. This code attempts to fetch, merge, and display using SwiftUI the contents of five news feeds.

If any of the fetches throws an error the whole group will throw an error and end, but if a fetch somehow succeeds while ending up with an empty array it means our data quota has run out and we should stop trying any other feed fetches.

Here's the code:

```
struct NewsStory: Identifiable, Decodable {
    let id: Int
    let title: String
    let strap: String
    let url: URL
}

struct ContentView: View {
    @State private var stories = [NewsStory]()

    var body: some View {
        NavigationStack {
            List(stories) { story in
                VStack(alignment: .leading) {
                    Text(story.title)
                        .font(.headline)

                    Text(story.strap)
                }
            }
            .navigationTitle("Latest News")
        }
        .task {
            await loadStories()
        }
    }

    func loadStories() async {
        do {
            try await withThrowingTaskGroup(of: [NewsStory].self) { group in
                for i in 1...5 {
                    group.addTask {
                        let url = URL(string: "https://hws.dev/news-\(i).json")!
                        let (data, _) = try await URLSession.shared.data(from: url)
                        try Task.checkCancellation()
                        return try JSONDecoder().decode([NewsStory].self, from: data)
                    }
                }

                for try await result in group {
                    if result.isEmpty {
                        group.cancelAll()
                    } else {
                        stories.append(contentsOf: result)
                    }
                }

                stories.sort { $0.id < $1.id }
            }
        } catch {
            print("Failed to load stories: \(error.localizedDescription)")
        }
    }
}
```

Download this as an Xcode project

As you can see, that calls `cancelAll()` as soon as any feed sends back an empty array, thus aborting all remaining fetches. Inside the child tasks there is an explicit call to `Task.checkCancellation()`, but the `data(from:)` also runs check for cancellation to avoid doing unnecessary work.

The other way task groups get cancelled is if one of the tasks throws an uncaught error. We can write a simple test for this by creating two tasks inside a group, both of which sleep for a little time. The first task will sleep for 1 second then throw an example error, whereas the second will sleep for 2 seconds then print the value of `Task.isCancelled`.

Here's how that looks:

```swift
enum ExampleError: Error {
    case badURL
}

func testCancellation() async {
    do {
        try await withThrowingTaskGroup(of: Void.self) { group in
            group.addTask {
                try await Task.sleep(for: .seconds(1))
                throw ExampleError.badURL
            }

            group.addTask {
                try await Task.sleep(for: .seconds(2 ))
                print("Task is cancelled: \(Task.isCancelled)")
            }

            try await group.next()
        }
    } catch {
        print("Error thrown: \(error.localizedDescription)")
    }
}

await testCancellation()
```

Download this as an Xcode project

Note: Just throwing an error inside `addTask()` isn't enough to cause other tasks in the group to be cancelled – this only happens when you access the value of the throwing task using `next()` or when looping over the child tasks. This is why the code sample above specifically waits for the result of a task, because doing so will cause `ExampleError.badURL` to be rethrown and cancel the other task.

Calling `addTask()` on your group will unconditionally add a new task to the group, even if you have already cancelled the group. If you want to avoid adding tasks to a cancelled group, use the `addTaskUnlessCancelled()` method instead – it works identically except will do nothing if called on a cancelled group. Calling `addTaskUnlessCancelled()` returns a Boolean that will be true if the task was successfully added, or false if the task group was already cancelled.

40. How to handle different result types in a task group

Each task in a Swift task group must return the same type of data as all the other tasks in the group, which is often problematic – what if you need one task group to handle several different types of data?

In this situation you should consider using `async let` for your concurrency if you can, because every `async let` expression can return its own unique data type. So, the first might result in an array of strings, the second in an integer, and so on, and once you've awaited them all you can use them however you please.

However, if you *need* to use task groups – for example if you need to create your tasks in a loop – then there is a solution: create an enum with associated values that wrap the underlying data you want to return.

Using this approach, each of the tasks in your group still return a single data type – one of the cases from your enum – but *inside* those cases you can place the unique data types you're actually using.

This is best demonstrated with some example code, but because it's quite a lot I'm going to add inline comments so you can see what's going on:

```
// A struct we can decode from JSON, storing one message from a contact.
struct Message: Decodable {
    let id: Int
    let from: String
    let message: String
}

// A user, containing their name, favorites list, and messages array.
struct User {
    let username: String
    let favorites: Set<Int>
    let messages: [Message]
}

// A single enum we'll be using for our tasks, each containing a different associated value.
enum FetchResult {
    case username(String)
    case favorites(Set<Int>)
    case messages([Message])
}

func loadUser() async {
    // Each of our tasks will return one FetchResult, and the whole group will send back a User.
    let user = await withThrowingTaskGroup(of: FetchResult.self) { group in
        // Fetch our username string
        group.addTask {
            let url = URL(string: "https://hws.dev/username.json")!
            let (data, _) = try await URLSession.shared.data(from: url)
            let result = String(decoding: data, as: UTF8.self)

            // Send back FetchResult.username, placing the string inside.
            return .username(result)
        }

        // Fetch our favorites set
        group.addTask {
            let url = URL(string: "https://hws.dev/user-favorites.json")!
            let (data, _) = try await URLSession.shared.data(from: url)
            let result = try JSONDecoder().decode(Set<Int>.self, from: data)

            // Send back FetchResult.favorites, placing the set inside.
            return .favorites(result)
        }

        // Fetch our messages array
        group.addTask {
            let url = URL(string: "https://hws.dev/user-messages.json")!
            let (data, _) = try await URLSession.shared.data(from: url)
            let result = try JSONDecoder().decode([Message].self, from: data)

            // Send back FetchResult.messages, placing the message array inside
            return .messages(result)
        }

        // At this point we've started all our tasks,
        // so now we need to stitch them together into
        // a single User instance. First, we set
        // up some default values:
        var username = "Anonymous"
        var favorites = Set<Int>()
        var messages = [Message]()
```

```
        // Now we read out each value, figure out
        // which case it represents, and copy its
        // associated value into the right variable.
        do {
            for try await value in group {
                switch value {
                case .username(let value):
                    username = value
                case .favorites(let value):
                    favorites = value
                case .messages(let value):
                    messages = value
                }
            }
        } catch {
            // If any of the fetches went wrong, we might
            // at least have partial data we can send back.
            print("Fetch at least partially failed; sending back what we have so far. \(error.localizedDescription)")
        }

        // Send back our user, either filled with
        // default values or using the data we
        // fetched from the server.
        return User(username: username, favorites: favorites, messages: messages)
    }

    // Now do something with the finished user data.
    print("User \(user.username) has \(user.messages.count) messages and \(user.favorites.count) favorites.")
}

await loadUser()
```

Download this as an Xcode project

I know it's a lot of code, but really it boils down to two things:

So, it's not impossible to handle heterogeneous results in a task group, it just requires a little extra thinking.

41. How to discard results in a task group

Swift has a special task group type called a *discarding* task group, which has a very specialized function: tasks that complete are automatically discarded and destroyed, rather than us needing to wait for them manually by calling `next()` or similar. This is particularly important for very long-running tasks, such as servers, where waiting for one task to complete is impractical.

We can try out a small example to let you see the problem. First, here's a simple `AsyncSequence` that continuously generates random numbers:

```
struct RandomGenerator: AsyncSequence, AsyncIteratorProtocol {
    mutating func next() async -> Int? {
        try? await Task.sleep(for: .seconds(0.001))
        return Int.random(in: 1...Int.max)
    }

    func makeAsyncIterator() -> Self {
        self
    }
}
```

In your own code, that might be a server that continuously accepts connections, or a filesystem watcher that continuously scans for updates.

Now we could make some test code that waits for a new number to come in, and passes it to a task group child to be processed, like this:

```
struct RandomGenerator: AsyncSequence, AsyncIteratorProtocol {
    mutating func next() async -> Int? {
        try? await Task.sleep(for: .seconds(0.001))
        return Int.random(in: 1...Int.max)
    }

    func makeAsyncIterator() -> Self {
        self
    }
}

let generator = RandomGenerator()

await withTaskGroup(of: Void.self) { group in
    for await newNumber in generator {
        group.addTask {
            print(newNumber)
        }
    }
}
```

Download this as an Xcode project

Here we're just printing the number, but if this were a server listening for new connections then the task we added would respond to those connections by serving up data.

Notice how we're saying our task group child type is `Void.self`, which means we're explicitly saying these tasks will return no values. However, our code never actually waits for tasks to complete, so they sit there, building up slowly – if you run that code back you'll it leaks about 0.5MB memory a second.

Tip: I added a tiny delay in our generator to avoid your computer being overwhelmed. I would recommend against removing it!

To fix this, we would need to await the result of the child tasks – when the data for a particular connection has been fully sent, for example, that task can then be destroyed.

However, that creates a different problem: if all the connections are currently being processed (i.e., if their tasks are currently active), then waiting for a task to finish will take some time, during which no new connections will be accepted.

This is where discarding task groups come in: you never need to wait for their result, and in fact *can't* wait for their result, because they are designed to be automatically destroyed on completion.

So, in our sample code we just need to replace this code:

```
await withTaskGroup(of: Void.self) { group in
```

With this code:

```
await withDiscardingTaskGroup { group in
```

And now our code won't leak memory any more, because old tasks will automatically be destroyed. Here's how the full code looks:

```
struct RandomGenerator: AsyncSequence, AsyncIteratorProtocol {
    mutating func next() async -> Int? {
        try? await Task.sleep(for: .seconds(0.001))
        return Int.random(in: 1...Int.max)
    }

    func makeAsyncIterator() -> Self {
        self
    }
}

let generator = RandomGenerator()

await withDiscardingTaskGroup { group in
    for await newNumber in generator {
        group.addTask {
            print(newNumber)
        }
    }
}
```

Download this as an Xcode project

Discarding task groups also have a *throwing* discarding task group, made using `withThrowingDiscardingTaskGroup()`.

42. Whats the difference between async let tasks and task groups

Swift's `async let`, `Task`, and task groups all solve a similar problem: they allow us to create concurrency in our code so the system is able to run them efficiently. Beyond that, the way they work is quite different, and which you'll choose depends on your exact scenario.

To help you understand how they differ, and provide some guidance on where each one is a good idea, I want to walk through the key behaviors of each of them.

First, `async let` and `Task` are designed to create specific, individual pieces of work, whereas task groups are designed to run multiple pieces of work at the same time and gather the results. As a result, `async let` and `Task` have no way to express a dynamic amount of work that should run in parallel.

For example, if you had an array of URLs and wanted to fetch them all in parallel, convert them into arrays of weather readings, then average them to a single `Double`, task groups would be a great choice because you won't know ahead of time how many URLs are in your array. Trying to write this using `async let` or `Task` just wouldn't work, because you'd have to hard-code the exact number of `async let` lines rather than just loop over an array.

Second, task groups automatically let us process results from child tasks in the order they complete, rather than in an order we specify. For example, if we wanted to fetch five pieces of data, task groups allow us to use `group.next()` to read whichever of the five comes back first, whereas using `async let` and `Task` would require us to await values in a specific, fixed order.

That alone is a helpful feature of task groups, but in some situations it goes from helpful to *crucial*. For example, if you have three possible servers for some data and want to use whichever one responds fastest, task groups are perfect – you can use `addTask()` once for each server, then call `next()` only once to read whichever one responded fastest.

Third, although all three forms of concurrency will automatically be marked as cancelled if their parent task is cancelled, only `Task` and task group can be cancelled directly, using `cancel()` and `cancelAll()` respectively. There is no equivalent for `async let`.

Fourth, because `async let` doesn't give us a handle to the underlying task it creates for us, it's not possible to pass that task elsewhere – we can't start an `async let` task in one function then pass that task to a different function. On the other hand, if you create a task that returns a string and never throws an error, you can pass that `Task<String, Never>` object around as needed.

And finally, although task groups *can* work with heterogeneous results – i.e., child tasks that return different types of data – it takes the extra work of making an enum to wrap the data. `async let` and `Task` do not suffer from this problem because they always return a single result type, so each result can be different.

By sheer volume of advantages you might think that `async let` is clearly much less useful than both `Task` and task groups, but not all those points carry equal weight in real-world code. In practice, I would suggest you're likely to:

- Use `async let` the most; it works best when there is a fixed amount of work to do.
- Use `Task` for some places where `async let` doesn't work, such as passing an incomplete value to a function.
- Use task groups least commonly, or at least use them *directly* least commonly – you might build other things on top of them.

I find that order is pretty accurate in practice, for a number of reasons:

So, again I would recommend you start with `async let`, move to `Task` if needed, then go to task groups only if there's something specific they offer that you need.

43. How to make async command line tools and scripts

If you're writing a command-line tool with Swift, you can use async code in two ways: if you're using main.swift you can immediately make and use `async` functions as normal, and if you aren't using main.swift you can use the `@main` attribute to launch your app into an async context immediately.

Either way, keep in mind that you should wait for your work to complete before allowing your program to terminate, otherwise it might not complete.

Let's try an example using both approaches. First, if you're using main.swift, you can just start using `await` and similar like this:

```
let url = URL(string: "https://hws.dev/users.csv")!

for try await line in url.lines {
    print("Received user: \(line)")
}
```

If you aren't using main.swift and instead prefer to use the `@main` attribute, you should first create the static `main()` method as you normally would with `@main`, then add `async` to it. You can optionally also add `throws` if you don't intend to handle errors there.

So, here's the previous example rewritten using `@main`:

```
@main
struct UserFetcher {
    static func main() async throws {
        let url = URL(string: "https://hws.dev/users.csv")!

        for try await line in url.lines {
            print("Received user: \(line)")
        }
    }
}
```

Download this as an Xcode project

Behind the scenes, Swift will automatically create a new task in which it runs your `main()` method, then terminate the program when that task finishes.

Tip: Just like using the `@main` attribute with a synchronous `main()` method, you should not include a main.swift file in your command-line project.

44. How to create and use task local values

Swift lets us attach metadata to a task using *task-local values*, which are small pieces of information that any code inside a task can read.

For example, we can read `Task.isCancelled` to see whether the current task is cancelled or not, but that's not a true static property – it's scoped to the current task, rather than shared across all tasks. This is the power of task-local values: the ability to create static-like properties inside a task.

Important: Most people will not want to use task-local values – if you're just curious you're welcome to read on and explore how task-local values work, but honestly they are useful in only a handful of very specific circumstances and if you find them complex I wouldn't worry too much.

Task-local values are analogous to thread-local values in an old-style multithreading environment: we attach some metadata to our task, and any code running inside that task can read that data as needed.

Swift's implementation is carefully scoped so that you create contexts where the data is available, rather than just injecting it directly into the task, which makes it possible to adjust your metadata over time. However, *inside* that context all code is able to read your task-local values, regardless of how it's used.

Using task-local values happens in three steps:

Inside the task-local scope, any time you read `YourType.yourProperty` you will receive the *task-local* value for that property – it's not a regular static property that has a single value shared between all parts of your program, but instead it can return a different value depending on which task tries to read it.

To demonstrate task-local values in action, I want to give you two examples: the first is a simple toy example that demonstrates the code required to use them and how they work, but the second is a more real-world example that's actually useful.

First, our simple example. This will create a `User` enum with a `id` property that is marked `@TaskLocal`, then it will launch a couple of tasks with different values for that user ID. Each task will do exactly the same thing: print the user ID, sleep for a small amount of time, then print the user ID again, which will allow you to see both tasks running at the same time while having their own unique task-local user ID.

Here's the code:

```
enum User {
    @TaskLocal static var id = "Anonymous"
}

@main
struct App {
    static func main() async throws {
        let first = Task {
            try await User.$id.withValue("Piper") {
                print("Start of task: \(User.id)")
                try await Task.sleep(for: .seconds(1))
                print("End of task: \(User.id)")
            }
        }

        let second = Task {
            try await User.$id.withValue("Alex") {
                print("Start of task: \(User.id)")
                try await Task.sleep(for: .seconds(1))
                print("End of task: \(User.id)")
            }
        }

        print("Outside of tasks: \(User.id)")
        try await first.value
        try await second.value
    }
}
```

Download this as an Xcode project

When that code runs it will print:

- Outside of tasks: Anonymous
- Start of task: Piper
- Start of task: Alex
- End of task: Piper
- End of task: Alex

Of course, because the two tasks run independently of each other you might also find that the order of Piper and Alex switch. The important thing is that each task has its own value for `User.id` even as they overlap, and code outside the task will continue to use the original value.

As you can see, Swift makes it impossible to forget about a task-local value you've set, because it only exists for the work inside `withValue()`. This scoping approach also means it's possible to nest multiple task locals as needed, and you can even shadow task locals – start a scope for one, do some work, then start another nested scope for that same property. so that it temporarily has a different value.

In real-world code, task-local values are useful for places where you need to repeatedly pass values around inside your tasks – values that need to be shared within the task, but not across your whole program like a singleton might be.

For example, the Swift Evolution proposal for task-local values (https://github.com/apple/swift-evolution/blob/main/proposals/0311-task-locals.md) suggests examples such as tracing, mocking, progress monitoring, and more.

As a more complex example, we could create a simple `Logger` struct that writes out messages depending on the current level of logging: debug being the lowest log level, then info, warn, error, and finally fatal at the highest level.

If we make the log level – which messages to print – be a task-local value, then each of our tasks can have whatever level of logging they want, regardless of what other tasks are doing.

To make this work we need three things:

On top of that, we need a couple more things to actually demonstrate the logger in action: a `fetch()` method that downloads data from a URL and creates various logging messages, and a couple of tasks that call `fetch()` with different task-local log settings so we can see exactly how it all works.

Here's the code:

```
// Our five log levels, marked Comparable so we can use < and > with them.
enum LogLevel: Comparable {
    case debug, info, warn, error, fatal
}

struct Logger {
    // The log level for an individual task
    @TaskLocal static var logLevel = LogLevel.info

    // Make this struct a singleton
    private init() { }
    static let shared = Logger()

    // Print out a message only if it meets or exceeds our log level.
    func write(_ message: String, level: LogLevel) {
        if level >= Logger.logLevel {
            print(message)
        }
    }
}

@main
struct App {
    // Returns data from a URL, writing log messages along the way.
    static func fetch(url urlString: String) async throws -> String? {
        Logger.shared.write("Preparing request: \(urlString)", level: .debug)

        if let url = URL(string: urlString) {
            let (data, _) = try await URLSession.shared.data(from: url)
            Logger.shared.write("Received \(data.count) bytes", level: .info)
            return String(decoding: data, as: UTF8.self)
        } else {
            Logger.shared.write("URL \(urlString) is invalid", level: .error)
            return nil
        }
    }
```

```
    // Starts a couple of fire-and-forget tasks with different log levels.
    static func main() async throws {
        let first = Task {
            try await Logger.$logLevel.withValue(.debug) {
                try await fetch(url: "https://hws.dev/news-1.json")
            }
        }

        let second = Task {
            try await Logger.$logLevel.withValue(.error) {
                try await fetch(url: "")
            }
        }

        _ = try await first.value
        _ = try await second.value
    }
}
```

Download this as an Xcode project

When that runs you'll see "Preparing request: https://hws.dev/news-1.json" as the first task starts, then "URL is invalid" as the second task starts (I used an empty string rather than a real URL), then "Received 8075 bytes" as the first task finishes downloading its data.

So, here our `fetch()` method doesn't even need to know that a task-local value is being used – it just calls the `Logger` singleton, which in turn refers to the task-local value.

To finish up, I want to leave you with a few important tips for using task-local values:

And finally, one important quote from the Swift Evolution proposal for this feature: "please be careful with the use of task-locals and don't use them in places where plain-old parameter passing would have done the job." Put more plainly, if task locals are the answer, there's a very good chance you're asking the wrong question.

45. How to run tasks using swiftuis task modifier

SwiftUI provides a `task()` modifier that starts a new task as soon as a view appears, and automatically cancels the task when the view disappears. This is sort of the equivalent of starting a task in `onAppear()` then cancelling it `onDisappear()`, although `task()` has an extra ability to track an identifier and restart its task when the identifier changes.

In the simplest scenario – and probably the one you're going to use the most – `task()` is the best way to load your view's initial data, which might be loaded from local storage or by fetching and decoding a remote URL.

Important: Because all SwiftUI views are automatically run on the main actor, the tasks they launch will also automatically run on the main actor until you move them elsewhere.

For example, this downloads data from a server and decodes it into an array for display in a list:

```
struct Message: Decodable, Identifiable {
    let id: Int
    let user: String
    let text: String
}

struct ContentView: View {
    @State private var messages = [Message]()

    var body: some View {
        NavigationStack {
            List(messages) { message in
                VStack(alignment: .leading) {
                    Text(message.user)
                        .font(.headline)

                    Text(message.text)
                }
            }
            .navigationTitle("Inbox")
            .task {
                await fetchData()
            }
        }
    }

    func fetchData() async {
        do {
            let url = URL(string: "https://hws.dev/inbox.json")!
            let (data, _) = try await URLSession.shared.data(from: url)
            messages = try JSONDecoder().decode([Message].self, from: data)
        } catch {
            messages = [
                Message(id: 0, user: "Failed to load inbox.", text: "Please try again later.")
            ]
        }
    }
}
```

Download this as an Xcode project

Important: The `task()` modifier is a great place to load the data for your SwiftUI views. Remember, they can be recreated many times over the lifetime of your app, so you should avoid putting this kind of work into their initializers if possible.

A more advanced usage of `task()` is to attach some kind of `Equatable` identifying value – when that value changes SwiftUI will automatically cancel the previous task and create a new task with the new value. This might be some shared app state, such as whether the user is logged in or not, or some local state, such as what kind of filter to apply to some data.

As an example, we could upgrade our messaging view to support both an Inbox and a Sent box, both fetched and decoded using the same `task()` modifier. By setting the message box type as the identifier for the task with `.task(id: selectedBox)`, SwiftUI will automatically update its message list every time the selection changes.

Here's how that looks in code:

```
struct Message: Decodable, Identifiable {
    let id: Int
    let user: String
    let text: String
}

// Our content view is able to handle two kinds of message box now.
struct ContentView: View {
    @State private var messages = [Message]()
    @State private var selectedBox = "Inbox"
    let messageBoxes = ["Inbox", "Sent"]

    var body: some View {
        NavigationStack {
            List(messages) { message in
                VStack(alignment: .leading) {
                    Text(message.user)
                        .font(.headline)

                    Text(message.text)
                }
            }
            .navigationTitle(selectedBox)

            // Our task modifier will recreate its fetchData() task whenever selectedBox changes
            .task(id: selectedBox) {
                await fetchData()
            }
            .toolbar {
                // Switch between our two message boxes
                Picker("Select a message box", selection: $selectedBox) {
                    ForEach(messageBoxes, id: \.self, content: Text.init)
                }
                .pickerStyle(.segmented)
            }
        }
    }

    // This is almost the same as before, but now loads the selectedBox JSON file rather than always loading the in
    func fetchData() async {
        do {
            let url = URL(string: "https://hws.dev/\(selectedBox.lowercased()).json")!
            let (data, _) = try await URLSession.shared.data(from: url)
            messages = try JSONDecoder().decode([Message].self, from: data)
        } catch {
            messages = [
                Message(id: 0, user: "Failed to load message box.", text: "Please try again later.")
            ]
        }
    }
}
```

Download this as an Xcode project

Tip: That example uses the shared `URLSession`, which means it will cache its responses and so load the two inboxes only once. If that's what you want you're all set, but if you want it to always fetch the files make sure you create your own session configuration and disable caching.

One particularly interesting use case for `task()` is with `AsyncSequence` collections that continuously generate values. This might be a server that maintains an open connection while sending fresh content, it might be the `URLWatcher` example we looked at previously, or perhaps just a local value.

For example, we could write a simple random number generator that regularly emits new random numbers – with the `task()` modifier we can constantly watch that for changes, and stream the results into a SwiftUI view.

To bring this example to life, we're going to add one more thing: the random number generator will print a message every time a number is generated, and the resulting number list will be shown inside a detail view. Both of these are done so you can see how `task()` automatically cancels its work: the numbers will automatically start streaming when the detail view is shown, and stop streaming when the view is dismissed.

Here's the code:

```
// A simple random number generator sequence
struct NumberGenerator: AsyncSequence, AsyncIteratorProtocol {
    let range: ClosedRange<Int>
    let delay: Double = 1

    mutating func next() async -> Int? {
```

```
            // Make sure we stop emitting numbers when our task is cancelled
            while Task.isCancelled == false {
                try? await Task.sleep(for: .seconds(delay))
                print("Generating number")
                return Int.random(in: range)
            }

            return nil
        }

        func makeAsyncIterator() -> NumberGenerator {
            self
        }
    }

    // This exists solely to show DetailView when requested.
    struct ContentView: View {
        var body: some View {
            NavigationStack {
                NavigationLink("Start Generating Numbers") {
                    DetailView()
                }
            }
        }
    }

    // This generates and displays all the random numbers we've generated.
    struct DetailView: View {
        @State private var numbers = [String]()
        let generator = NumberGenerator(range: 1...1000)

        var body: some View {
            List(numbers, id: \.self, rowContent: Text.init)
                .task {
                    await generateNumbers()
                }
        }

        func generateNumbers() async {
            for await number in generator {
                numbers.insert("\(numbers.count + 1). \(number)", at: 0)
            }
        }
    }
```

Download this as an Xcode project

Notice how the `generateNumbers()` method at the end doesn't actually have any way of exiting? That's because it will exit automatically when `generator` stops returning values, which will happen when the task is cancelled, and *that* will happen when `DetailView` is dismissed – it takes no special work from us.

Tip: The `task()` modifier accepts a `priority` parameter if you want fine-grained control over your task's priority. For example, use `.task(priority: .low)` to create a low-priority task.

46. Is it efficient to create many tasks

Previously I talked about the concept of *thread explosion*, which is when you create many more threads than CPU cores and the system struggles to manage them effectively. However, Swift's tasks are implemented very differently from threads, and so are significantly less likely to cause performance problems when used in large numbers, and in fact one of the Swift team who worked on it said that unless you're creating over 10,000 tasks it's not worth worrying about the impact of so many tasks.

That doesn't mean creating so many tasks is necessarily the best idea. You might think that's hard to do, but a task group calling `addTask()` inside a loop might create several hundred or even several *thousand* depending on what you were looping over. And that's okay. Again, even if you're creating well over 10,000 tasks it's not likely to cause a problem as long as you *know* that's what you're doing – if that's the architectural choice you're making after evaluating the alternative.

So, broadly speaking you should feel free to create as many tasks as you want, but if you ever find yourself creating tasks to transform elements inside huge arrays you might want to double-check your performance using something like Instruments.

47. What is an actor and why does swift have them

Swift's actors are conceptually like classes that are safe to use in concurrent environments. This safety is made possible because Swift automatically ensures no two pieces of code attempt to access an actor's data simultaneously – it is made impossible by the compiler, rather than requiring developers to write boilerplate code using systems such as locks.

In the following chapters we're going to explore more about how actors work and when you should use them, but here is the least you need to know:

As well as those, there is one more behavior of actors that lies at the center of their existence: if you're attempting to read a variable property or call a method on an actor, and you're doing it from outside the actor itself, you must do so asynchronously using `await`.

I'll explain why in a moment, but I want to show you a brief snippet of code first so you can see what I mean:

```
actor User {
    var score = 10

    func printScore() {
        print("My score is \(score)")
    }

    func copyScore(from other: User) async {
        score = await other.score
    }
}

let actor1 = User()
let actor2 = User()

await print(actor1.score)
print(await actor1.score)
await actor1.copyScore(from: actor2)
```

Download this as an Xcode project

You can see several things in action there:

The reason the `await` call is needed in `copyScore(from:)` is central to the reasons actors are needed at all.

You see, rather than just letting us poke around in an actor's mutable state, Swift silently translates that request into what is effectively a message that goes into the actor's message inbox: "please let me know your score as soon as you can."

If the actor is currently idle it can respond with the score straight away and our code continues no different from if we had used a class or a struct. But the actor might also have multiple other messages waiting in its inbox that it needs to deal with first, so our `score` request has to wait. Eventually our request makes it to the top of the inbox and it will be dealt with, and the `copyScore(from:)` method will continue.

This means several things:

In practice, the rule to remember is this: if you are writing code inside an actor's method, you can read other properties on that actor and call its synchronous methods without using `await`, but if you're trying to use that data from *outside* the actor `await` is required even for synchronous properties and methods. Think of situations where using `self` is possible: if you could `self` you don't need `await`.

Writing properties from outside an actor is not allowed, with or without `await`.

Of course, the real question here is why Swift needs actors at all – what's their fundamental purpose? And the answer is straightforward: if you ever need to make sure that access to some object is restricted to a single active task at a time, actors are perfect.

This is more common than you might think – yes, UI work should be restricted to the main thread, but you probably also want to restrict database access so that you can't write conflicting data, for example, and so SwiftData has model actors.

There are also times when simultaneous concurrent access to data can cause *data races* – when the outcome of your work depends on the order in which tasks complete. These errors are particularly nasty to find and fix, but with actors such data races become impossible.

However, actor functions are *reentrant*, which is a computer science term that means one task can start while another one is still running. I know that sounds like it breaks the rules I laid out above, but it's one of the subtle nuances of actors you'll need to understand in order to use them effectively.

Tip: Creating an instance of an actor has no extra performance cost compared to creating an instance of a class; the only performance difference comes when trying to access the protected state of an actor, which might trigger task suspension.

48. How to create and use an actor in swift

Creating and using an actor in Swift takes two steps: create the type using `actor` rather than `class` or `struct`, then use `await` when accessing its properties or methods externally. Swift takes care of everything else for us, including ensuring that properties and methods must be accessed safely.

Let's look at a simple example: authenticating a user using a remote server. We'd start by creating our actor:

```
actor AuthenticationManager {
    var token: String?

    var isAuthenticated: Bool {
        token != nil
    }

    func authenticate(username: String, password: String) async throws {
        // Simulate authenticating a user.
        let url = URL(string: "https://hws.dev/authenticate")!
        let (data, _) = try await URLSession.shared.data(from: url)
        token = String(decoding: data, as: UTF8.self)
    }
}
```

And now we could create an instance of that actor and use it in two concurrent tasks, like this:

```
actor AuthenticationManager {
    var token: String?

    var isAuthenticated: Bool {
        token != nil
    }

    func authenticate(username: String, password: String) async throws {
        // Simulate authenticating a user.
        let url = URL(string: "https://hws.dev/authenticate")!
        let (data, _) = try await URLSession.shared.data(from: url)
        token = String(decoding: data, as: UTF8.self)
    }
}

let manager = AuthenticationManager()

let first = Task {
    do {
        // Attempt to log in.
        try await manager.authenticate(username: "twostraws", password: "samoyed123")

        // Get the user's authentication token.
        if let token = await manager.token {
            print("User token: \(token)")
        }
    } catch {
        print("Authentication failed: \(error.localizedDescription)")
    }
}

let second = Task {
    // Attempt to access the authentication status elsewhere.
    let authenticated = await manager.isAuthenticated
    print("Second task check: \(authenticated)")
}

await first.value
await second.value
```

Download this as an Xcode project

We don't know or care in what order those tasks run, or even if they run in parallel – using an actor makes sure they can be accessed in two separate tasks, because it automatically ensures access to properties like `token` and `isAuthenticated` are handled separately.

If we didn't have an actor here – if we had used `class AuthenticationManager`, for example – then we would need to solve those two problems ourselves. It's not *hard*, at least not in a simple way, but it's error-prone and boring to do, so it's great to be able to hand this work over to the Swift compiler to do for us.

The canonical example of why data races are problematic – the one that is often taught in computer science degrees – is about *bank accounts*, because here data races can result in serious real-world problems.

To see why, here's an example `BankAccount` class that handles sending and receiving money:

```
class BankAccount {
    var balance: Decimal

    init(initialBalance: Decimal) {
        balance = initialBalance
    }

    func deposit(amount: Decimal) {
        balance = balance + amount
    }

    func transfer(amount: Decimal, to other: BankAccount) {
        // Check that we have enough money to pay
        guard balance >= amount else { return }

        // Subtract it from our balance
        balance = balance - amount

        // Send it to the other account
        other.deposit(amount: amount)
    }
}

let firstAccount = BankAccount(initialBalance: 500)
let secondAccount = BankAccount(initialBalance: 0)
firstAccount.transfer(amount: 500, to: secondAccount)
```

Download this as an Xcode project

That's a class, so Swift won't do anything to stop us from accessing the same piece of data multiple times. So, what could actually happen here?

Well, in the worst case two parallel calls to `transfer()` would be called on the same `BankAccount` instance, and the following would occur:

Do you see the problem there? Well, what happens if the account we're transferring from contains $100, and we're asked to transfer $80 to the other account? If we follow the steps above, both calls to `transfer()` will happen in parallel and see that there was enough for the transfer to take place, then both will transfer the money across.

The end result is that our check for sufficient funds wasn't useful, and one account ends up with -$60 – something that might incur fees, or perhaps not even be allowed depending on the type of account they have.

If we switch this type to be an actor, that problem goes away. This means using `actor BankAccount` rather than `class BankAccount`, but also using `async` and `await` because we can't directly call `deposit()` on the other bank account and instead need to post the request as a message to be executed later.

Here's how that looks:

```
actor BankAccount {
    var balance: Decimal

    init(initialBalance: Decimal) {
        balance = initialBalance
    }

    func deposit(amount: Decimal) {
        balance = balance + amount
    }

    func transfer(amount: Decimal, to other: BankAccount) async {
        // Check that we have enough money to pay
        guard balance > amount else { return }

        // Subtract it from our balance
        balance = balance - amount

        // Send it to the other account
        await other.deposit(amount: amount)
    }
}
```

```
let firstAccount = BankAccount(initialBalance: 500)
let secondAccount = BankAccount(initialBalance: 0)
await firstAccount.transfer(amount: 500, to: secondAccount)
```

Download this as an Xcode project

With that change, our bank accounts can no longer fall into negative values by accident, which avoids a potentially nasty result.

In other places, actors can prevent bizarre results that ought to be impossible. For example, what would happen if our example was a basketball team rather than a bank account, and we were transferring players rather than money?

Without actors we could end up in the situation where we transfer the same player twice – Team A would end up without them, and Team B would have them twice!

49. Understanding actor initialization

Swift's actors run on their own executor most of the time, so they manage how and where their work is done. However, during the actor's initialization its executor isn't ready to take on work. So, the Swift team are working on a rule to keep everything safe: they want to make actors with an async initializer automatically move to the actor's executor when its properties are all initialized.

Here's how that looks in code:

```
actor Actor {
    var name: String

    init(name: String) async {
        print(name)
        self.name = name
        print(name)
    }
}

let actor = await Actor(name: "Meryl")
```

Download this as an Xcode project

At the time of writing this feature isn't fully implemented, but it's the team's goal to have this feature working in the future. So you should be aware that in code like the above the two `print()` calls might execute on entirely different threads – in effect, there's an implicit actor hop as soon as `self.name` is assigned.

50. How to make function parameters isolated

Any properties and methods that belong to an actor are isolated to that actor, but you can make *external* functions isolated to an actor if you want. This allows the function to access actor-isolated state as if it were inside that actor, without needing to use `await`.

Here's a simple example so you can see what I mean:

```
actor DataStore {
    var username = "Anonymous"
    var friends = [String]()
    var highScores = [Int]()
    var favorites = Set<Int>()

    init() {
        // load data here
    }

    func save() {
        // save data here
    }
}

func debugLog(dataStore: isolated DataStore) {
    print("Username: \(dataStore.username)")
    print("Friends: \(dataStore.friends)")
    print("High scores: \(dataStore.highScores)")
    print("Favorites: \(dataStore.favorites)")
}

let data = DataStore()
await debugLog(dataStore: data)
```

Download this as an Xcode project

That creates a `DataStore` actor with various properties plus a couple of placeholder methods, then creates a `debugLog()` function that prints those *without* using `await` – they can be accessed directly.

Notice the addition of the `isolated` keyword in the function signature; that's what allows this direct access, and it even allows the function to *write* to those properties too.

Using `isolated` like this does *not* bypass any of the underlying safety or implementation of actors – there can still only be one thread accessing the actor at any one time. What we've done just pushes that access out by a level, because now the whole function must be run on that actor rather than just individual lines inside it. In practice, this means `debugLog(dataStore:)` needs to be called using `await`.

This approach has an important side effect: because the whole function is now isolated to the actor, it must be called using `await` even though it isn't marked as async. This makes the function itself a single potential suspension point rather than individual accesses to the actor being suspension points.

In case you were wondering, you can't have two isolation parameters, because it wouldn't really make sense – which one is executing the function?

51. How to make parts of an actor nonisolated

All methods and mutable properties inside an actor are isolated to that actor by default, which means they cannot be accessed directly from code that's external to the actor. Access to constant properties is automatically allowed because they are inherently safe from race conditions, but if you want you can make some methods excepted by using the `nonisolated` keyword.

Actor methods that are non-isolated can access other non-isolated state, such as constant properties or other methods that are marked non-isolated. However, they *cannot* directly access *isolated state* like an isolated actor method would; they need to use `await` instead.

To demonstrate non-isolated methods, we could write a `User` actor that has three properties: two constant strings for their username and password, and a variable Boolean to track whether they are online. Because `password` is constant, we could write a non-isolated method that returns the hash of that password using CryptoKit, like this:

```
import CryptoKit
import Foundation

actor User {
    let username: String
    let password: String
    var isOnline = false

    init(username: String, password: String) {
        self.username = username
        self.password = password
    }

    nonisolated func passwordHash() -> String {
        let passwordData = Data(password.utf8)
        let hash = SHA256.hash(data: passwordData)
        return hash.compactMap { String(format: "%02x", $0) }.joined()
    }
}

let user = User(username: "twostraws", password: "s3kr1t")
print(user.passwordHash())
```

Download this as an Xcode project

I'd like to pick out a handful of things in that code:

Non-isolated methods *won't* help when it comes to protocol conformance – things like `Codable` and `Equatable` are strictly synchronous only at the time of writing.

52. How to use mainactor to run code on the main queue

`@MainActor` is a global actor that uses the main queue for executing its work. In practice, this means methods or types marked with `@MainActor` can (for the most part) safely modify the UI because it will always be running on the main queue, and calling `MainActor.run()` will push some custom work of your choosing to the main actor, and thus to the main queue.

At the simplest level both of these features are straightforward to use, but as you'll see there's a lot of complexity behind them.

First, let's look at using `@MainActor`, which automatically makes a single method or all methods on a type run on the main actor. This is particularly useful for any types that exist to update your user interface, such as `@Observable` or `ObservableObject` classes.

For example, we could create an `@Observable` class with two properties, and because they will both update the UI we would mark the whole class with `@MainActor` to ensure these UI updates always happen on the main actor no matter where they are triggered from:

```
@Observable @MainActor
class AccountViewModel {
    var username = "Anonymous"
    var isAuthenticated = false
}
```

Or if you were using the old `ObservableObject` protocol, it would be this:

```
@MainActor
class AccountViewModel: ObservableObject {
    @Published var username = "Anonymous"
    @Published var isAuthenticated = false
}
```

Now, in Xcode 16 and later SwiftUI was updated so that all structs that conform to `View` are automatically and entirely run on the main actor, which makes all this work a great deal easier. This is *not* attached to your deployment target – any iOS you compile using Xcode 16 or later benefits from this change.

Does that mean you don't need to explicitly add `@MainActor` to your observable classes? Well, no – there are still benefits to using `@MainActor` with these classes, not least if they are using `async`/`await` to do their own asynchronous work such as downloading data from a server.

So, my recommendation is simple: even though SwiftUI forces main-actor-ness for all its views, it's still a good idea to add the `@MainActor` attribute to all your observable classes to be absolutely sure all UI updates happen on the main actor. If you need certain methods or computed properties to opt out of running on the main actor, use `nonisolated` as you would do with a regular actor.

Important: I've said it previously, but it's worth repeating: you should *not* attempt to use actors for your observable objects, because we must do all UI updates on the main actor rather than a custom actor.

The magic of `@MainActor` is that it automatically forces methods or whole types to run on the main actor, a lot of the time without any further work from us. Previously we needed to do it by hand, remembering to use code like `DispatchQueue.main.async()` or similar every place it was needed, but now the compiler does it for us automatically.

Be careful: `@MainActor` is really helpful to make code run on the main actor, but it's not *foolproof*. For example, if you have a `@MainActor` class then in theory all its methods will run on the main actor, but one of those methods could trigger code to run on a background task. For example, if you're using Face ID and call `evaluatePolicy()` to authenticate the user, the completion handler will be called on a background thread even though that code is still within the `@MainActor` class.

If you *do* need to spontaneously run some code on the main actor, you can do that by calling `MainActor.run()` and providing your work. This allows you to safely push work onto the main actor no matter where your code is currently running, like this:

```
func couldBeAnywhere() async {
    await MainActor.run {
        print("This is on the main actor.")
    }
}

await couldBeAnywhere()
```

Download this as an Xcode project

You can send back nothing from `run()` if you want, or send back a value like this:

```
func couldBeAnywhere() async {
    let result = await MainActor.run {
        print("This is on the main actor.")
        return 42
    }

    print(result)
}

await couldBeAnywhere()
```

Download this as an Xcode project

Even better, if that code was already running on the main actor then the code is executed immediately – it won't wait until the next run loop in the same way that `DispatchQueue.main.async()` would have done.

If you wanted the work to be sent off to the main actor *without* waiting for its result to come back, you can place it in a new task like this:

```
func couldBeAnywhere() {
    Task {
        await MainActor.run {
            print("This is on the main actor.")
        }
    }

    // more work you want to do
}

couldBeAnywhere()
```

Download this as an Xcode project

Or you can also mark your task's closure as being `@MainActor`, like this:

```
func couldBeAnywhere() {
    Task { @MainActor in
        print("This is on the main actor.")
    }

    // more work you want to do
}

couldBeAnywhere()
```

Download this as an Xcode project

This is particularly helpful when you're inside a synchronous context, so you need to push work to the main actor without using the `await` keyword.

Important: If your function is already running on the main actor, using `await MainActor.run()` will run your code immediately without waiting for the next run loop, but using `Task` as shown above *will* wait for the next run loop.

You can see this in action in the following snippet:

```
@MainActor @Observable
class ViewModel {
    func runTest() async {
        print("1")

        await MainActor.run {
            print("2")

            Task { @MainActor in
                print("3")
            }

            print("4")
        }

        print("5")
    }
}

let model = ViewModel()
await model.runTest()
try await Task.sleep(for: .seconds(0.1))
```

That marks the whole type as using the main actor, so the call to `MainActor.run()` will run immediately when `runTest()` is called. However, the inner `Task` will *not* run immediately, so the code will print 1, 2, 4, 5, 3.

Although it's possible to create your own global actors, I think you should probably avoid doing so until you've had sufficient chance to build apps using what you already have.

53. Who decides which actor code runs on

One common confusion with actors comes in deciding exactly where code runs, because just adding `@MainActor` and hoping for the best is rarely good enough.

A common problem is code like this:

```
Task { @MainActor in
    await downloadData()
}
```

That does three things at once:

The confusion here is simple: you might think `downloadData()` will run on the main actor, but from the code above we have no idea whether that will be true or not – it could run on any actor.

To understand why, remember the rule that every time you see the `await` keyword it marks a potential suspension point – when Swift reaches an `await`, it's free to transfer execution wherever it needs to.

So, perhaps `downloadData()` is defined like this:

```
func downloadData() async {
    // do work here
}
```

That hasn't specified where the work should be done, so Swift is free to choose – and it's very likely to do it away from the main actor.

Alternatively, perhaps it's defined like this:

```
@MainActor
func downloadData() async {
    // do work here
}
```

…in which case it will run on the main actor regardless. The same goes for if it belongs to an actor-isolated type, like this:

```
@MainActor
class DataFetcher {
    func downloadData() async {
        // do work here
    }
}
```

This isn't a bug, it's completely intentional: code that you want to run on the main actor or indeed any actor must be marked as such – the code being called gets to decide where it runs, rather than the caller.

Let's return to the problem code again:

```
Task { @MainActor in
    await downloadData()
}
```

If `downloadData()` isn't guaranteed to run on the main actor, what exactly is the `@MainActor` annotation doing? The answer is: nothing at all. Using `@MainActor in` with a task means "the body of this closure must run on the main actor, but any async code we call can run elsewhere."

So, synchronous code in the closure will definitely run on the main actor, as you can see here:

```
Task {
    print("This will be on the main actor")
    await downloadData()
    print("This will also ben on the main actor")
}
```

The mistake is thinking that `downloadData()` will also run on the main actor, because unless we see how it's defined we simply can't tell.

The rule to remember is this: async functions get to choose where they run, irrespective of how you call them.

54. Understanding how global actor inference works

Apple explicitly annotates many of its types as being `@MainActor`, including most UIKit types such as `UIView` and `UIButton`. However, for a while the Swift team dabbled with the idea of making some types gain main-actor-ness *implicitly* through a process called *global actor inference* – Swift applied `@MainActor` automatically based on a set of predetermined rules.

Global actor inference was enabled from Swift 5.5 through to Swift 5.10. It is disabled in Swift 6 language mode, so if you're building using Swift 6 language mode or later you can ignore all of the below.

Still here?

Okay, there are five rules for global actor inference, and I want to tackle them individually because although they start easy they get more complex.

First, if a class is marked `@MainActor`, all its subclasses are also automatically `@MainActor`. This follows the principle of least surprise: if you inherit from a `@MainActor` class it makes sense that your subclass is also `@MainActor`.

Second, if a method in a class is marked `@MainActor`, any overrides for that method are also automatically `@MainActor`. Again, this is a natural thing to expect – you overrode a `@MainActor` method, so the only safe way Swift can call that override is if it's also `@MainActor`.

Third, any struct or class using a property wrapper with `@MainActor` for its wrapped value will automatically be `@MainActor`. In SwiftUI, this makes `@StateObject` and `@ObservedObject` convey main-actor-ness on SwiftUI views that use them – if you use either of those two property wrappers in a SwiftUI view, the whole view becomes `@MainActor` too.

The fourth rule is where the difficulty ramps up a little: if a protocol declares a method as being `@MainActor`, any type that conforms to that protocol will have that same method automatically be `@MainActor` *unless* you separate the conformance from the method.

What this means is that if you make a type conform to a protocol with a `@MainActor` method, and add the required method implementation at the same time, it is implicitly `@MainActor`. However, if you separate the conformance and the method implementation, you need to add `@MainActor` by hand.

Here's that in code:

```
// A protocol with a single `@MainActor` method.
protocol DataStoring {
    @MainActor func save()
}

// A struct that does not conform to the protocol.
struct DataStore1 { }

// When we make it conform and add save() at the same time, our method is implicitly @MainActor.
extension DataStore1: DataStoring {
    func save() { } // This is automatically @MainActor.
}

// A struct that conforms to the protocol.
struct DataStore2: DataStoring { }

// If we later add the save() method, it will *not* be implicitly @MainActor so we need to mark it as such ourselve
extension DataStore2 {
    @MainActor func save() { }
}
```

As you can see, we need to explicitly use `@MainActor func save()` in `DataStore2` because the global actor inference does not apply there. Don't worry about forgetting it, though – Xcode will automatically check the annotation is there, and offer to add `@MainActor` if it's missing.

The fifth and final rule is most complex of all: if the whole protocol is marked with `@MainActor`, any type that conforms to that protocol will also automatically be `@MainActor` *unless* you put the conformance separately from the main type declaration, in which case only the methods are `@MainActor`.

In attempt to make this clear, here's what I mean:

```swift
// A protocol marked as @MainActor.
@MainActor protocol DataStoring {
    func save()
}

// A struct that conforms to DataStoring as part of its primary type definition.
struct DataStore1: DataStoring { // This struct is automatically @MainActor.
    func save() { } // This method is automatically @MainActor.
}

// Another struct that conforms to DataStoring as part of its primary type definition.
struct DataStore2: DataStoring { } // This struct is automatically @MainActor.

// The method is provided in an extension, but it's the same as if it were in the primary type definition.
extension DataStore2 {
    func save() { } // This method is automatically @MainActor.
}

// A third struct that does *not* conform to DataStoring in its primary type definition.
struct DataStore3 { } // This struct is not @MainActor.

// The conformance is added as an extension
extension DataStore3: DataStoring {
    func save() { } // This method is automatically @MainActor.
}
```

I realize that might sound obscure, but it makes sense if you put it into a real-world context. For example, let's say you were working with the `DataStoring` protocol we defined above – what would happen if you modified one of Apple's types so that it conformed to it?

If conformance to a `@MainActor` protocol retroactively made the whole of Apple's type `@MainActor` then you would have dramatically altered the way it worked, probably breaking all sorts of assumptions made elsewhere in the system.

If it's *your* type – a type you're creating from scratch in your own code – then you *can* add the protocol conformance as you make the type and therefore isolate the entire type to `@MainActor`, because it's your choice.

55. What is actor hopping and how can it cause problems

When a thread pauses work on one actor to start work on another actor instead, we call it *actor hopping*, and it will happen any time one actor calls another.

Behind the scenes, Swift manages a group of threads called the *cooperative thread pool*, creating as many threads as there are CPU cores so that we can't be hit by thread explosion. Actors guarantee that they can be running only one method at a time, but they don't care which thread they are running on – they will automatically move between threads as needed in order to balance system resources.

Actor hopping with the cooperative pool is fast – it will happen automatically, and we don't need to worry about it. However, the main thread is *not* part of the cooperative thread pool, which means actor code being run from the main actor will require a context switch, which will incur a performance penalty if done too frequently.

You can see the problem caused by frequent actor hopping in this toy example code:

```
actor NumberGenerator {
    var lastNumber = 1

    func getNext() -> Int {
        defer { lastNumber += 1 }
        return lastNumber
    }

    @MainActor func run() async {
        for _ in 1...100 {
            let nextNumber = await getNext()
            print("Loading \(nextNumber)")
        }
    }
}

let generator = NumberGenerator()
await generator.run()
```

Download this as an Xcode project

In that code, the `run()` method must take place on the main actor because it has the `@MainActor` attribute attached to it, however the `getNext()` method will run somewhere on the cooperative pool, meaning that Swift will need to perform frequent context switching from to and from the main actor inside the loop.

In practice, your code is more likely to look like this:

```
// An example piece of data we can show in our UI
struct User: Identifiable {
    let id: Int
}

// An actor that handles serial access to a database
actor Database {
    func loadUser(id: Int) -> User {
        // complex work to load a user from the database
        // happens here; we'll just send back an example
        User(id: id)
    }
}

// An object that handles updating our UI
@Observable @MainActor
class DataModel {
    var users = [User]()
    var database = Database()

    // Load all our users, updating the UI as each one
    // is successfully fetched
    func loadUsers() async {
        for i in 1...100 {
            let user = await database.loadUser(id: i)
            users.append(user)
        }
    }
```

```
}

// A SwiftUI view showing all the users in our data model
struct ContentView: View {
    @State private var model = DataModel()

    var body: some View {
        List(model.users) { user in
            Text("User \(user.id)")
        }
        .task {
            await model.loadUsers()
        }
    }
}
```

Download this as an Xcode project

When that runs, the `loadUsers()` method will run on the main actor, because the whole `DataModel` class must run there – it has been annotated with `@MainActor` to avoid publishing changes from a background thread.

However, the database's `loadUser()` method will run somewhere on the cooperative pool: it might run on thread 3 the first time it's called, thread 5 the second time, thread 8 the third time, and so on; Swift will take care of that for us.

This means when our code runs it will repeatedly hop to and from the main actor, meaning there's a significant performance cost introduced by all the context switching.

The solution here is to avoid all the switches by running operations in batches – hop to the cooperative thread pool once to perform all the actor work required to load many users, then process those batches on the main actor. The batch size could potentially load all users at once depending on your need, but even batch sizes of two would halve the context switches compared to individual fetches.

For example, we could rewrite our previous example like this:

```
struct User: Identifiable {
    let id: Int
}

actor Database {
    func loadUsers(ids: [Int]) -> [User] {
        // complex work to load users from the database
        // happens here; we'll just send back examples
        ids.map(User.init)
    }
}

@Observable @MainActor
class DataModel {
    var users = [User]()
    var database = Database()

    func loadUsers() async {
        let ids = Array(1...100)

        // Load all users in one hop
        let newUsers = await database.loadUsers(ids: ids)

        // Now back on the main actor, update the UI
        users.append(contentsOf: newUsers)
    }
}

struct ContentView: View {
    @State private var model = DataModel()

    var body: some View {
        List(model.users) { user in
            Text("User \(user.id)")
        }
        .task {
            await model.loadUsers()
        }
    }
}
```

Download this as an Xcode project

Notice how the SwiftUI view is identical – we're just rearranging our internal data access to be more efficient.

56. What is actor reentrancy and how can it cause problems

Actor-isolated functions are reentrant, which means it's possible for one piece of work to begin before a previous piece of work completes. This opens the possibility of subtle bugs, so if you intend to use actors you should read this chapter carefully.

First, you already know that whenever you see the `await` keyword it marks a potential suspension point: once your code resumes after the `await`, the state of your program might have changed. Actor reentrancy means this is true of actors too: if you use `await` inside an actor method, that whole piece of work might suspend, and potentially *another* piece of work will begin while the first task is suspended.

You can see actor reentrancy in action with this simple code:

```
actor Player {
    var name = "Anonymous"
    var score = 0

    func addToScore() {
        Task {
            score += 1
            try? await Task.sleep(for: .seconds(1))
            print("Score is now \(score)")
        }
    }
}

let player = Player()
await player.addToScore()
await player.addToScore()
await player.addToScore()

try? await Task.sleep(for: .seconds(1.1))
```

Download this as an Xcode project

In there we have one actor doing the same task three times, but each time it goes to sleep for a second using an `await` call.

And so, the execution will be something like this:

- `addToScore()` starts running, and adds 1 to `score`. At this point, `score` is 1.
- It then goes to sleep for one second, at which point the actor is free to start another task.
- `addToScore()` starts running a second time, adding another 1 to `score` making it 2.
- The second run then *also* goes to sleep, and again the actor is free to start new work.
- `addToScore()` starts running a third time, adding another 1 to `score` to make 3.
- The third run goes to sleep, just like the others.
- The first task finishes its sleep, at which it prints score with its current value of 3.
- The second task then finishes sleeping, and prints 3.
- The third task does the same.
- We have one final sleep for 1.1 seconds, to make sure the actor finishes all its work before the program run terminates.

So, when the code runs, it will print "Score is now 3" three times, because by the time all three tasks awaken that's what the `score` value will be.

Swift calls this process is called *interleaving*, and it's only possible wherever a suspension point is reached – regular synchronous code will run in its entirety without any suspension points, and therefore can't be hit by actor reentrance.

The real rule we're dealing with when using actors is this: no two tasks will run *in parallel*, meaning executing at same time. Several tasks could in theory be being handled by the actor right now, but no more than one can run

simultaneously.

This isn't a bug in actors, but instead an intentional performance solution because it guarantees that your actor will make some progress on work available to it. This in turn helps to avoid a problem known as *deadlocks*, where two tasks are unable to proceed because each is waiting for the other to release a resource they need to continue.

Important: If you both need to restrict access to state and also need to avoid actor reentrancy, it's tempting to add some kind of manual locking inside an actors. However, that's almost certainly a very bad idea because it's just not how actors were designed – creating your own separate serial queue is probably what you want.

57. Whats the difference between actors classes and structs

Swift provides four concrete nominal types for defining custom objects: actors, classes, structs, and enums. Each of these works a little differently from the others, but the first three might seem similar so it's worth spending a little time clarifying what they have in common and where their differences are.

Tip: Ultimately, which you use depends on the exact context you're working in, and you will need them all at some point.

Actors:

Classes:

Structs:

You might think the advantages of actors are such that they should be used everywhere classes are currently used, but that is a bad idea. Not only do you lose the ability for inheritance, but you'll also cause a huge amount of pain for yourself because every single external property access needs to use `await`.

However, there are certainly places where actors are a natural fit. For example, if you were previously creating serial queues to handle specific workflows, they can be replaced almost entirely with actors – while also benefiting from increased safety and performance. So, if you have some work that absolutely must work one at a time, such as accessing a database, then trying converting it into something like a database actor.

There is one area in particular where using actors rather than classes is going to cause problems, so I really can't say this clearly enough:

Do not use actors for your SwiftUI data models. You should use a class that uses the `@Observable` macro or conforms to the `ObservableObject` protocol instead. If needed, you can optionally also mark that class with `@MainActor` to ensure it does any UI work safely, but keep in mind that all SwiftUI views automatically run their code on the main actor.

If you desperately need to be able to carve off some async work safely, you can create a sibling actor – a separate actor that does not use `@MainActor`, but does not directly update the UI.

58. Important do not use an actor for your swiftui data models

Swift's actors allow us to share data in multiple parts of our app *without* causing problems with concurrency, because they automatically ensure two pieces of code cannot simultaneously access the actor's protected data.

Actors are an important addition to our toolset, and help us guarantee safe access to data in concurrent environments. However, if you've ever wondered "should I use an actor for my SwiftUI data?", let me answer that as clearly as I can: actors are a *really* bad choice for any data models you use with SwiftUI.

SwiftUI updates its user interface on the main actor, which means when we make a class use the `@Observable` macro or conform to `ObservableObject` we're agreeing that all our work will happen on the main actor. As an example, any time we modify an `@Published` property that *must* happen on the main actor, otherwise we'll be asking for changes to be made somewhere that isn't allowed.

Now think about what would happen if you tried to use a custom actor for your data. Not only would any data writes need to happen on that actor rather than the main actor (thus forcing the UI to update away from the main actor), but any data *reads* would need to happen there too – every time you tried to bind a string to a `TextField`, for example, you'd be asking Swift to simultaneously use the main actor and your custom actor, which doesn't make sense.

The correct solution here is to use a class that either uses the `@Observable` macro or conforms to `ObservableObject`, then annotate it with `@MainActor` to ensure it does any UI work safely. If you still find that you need to be able to carve off some async work safely, you can create a sibling actor – a separate actor that does not use `@MainActor`, but does not directly update the UI.

59. Introduction to testing swift concurrency

Swift's full range of concurrency features work great with unit tests using both the legacy XCTest framework and the new Swift Testing framework.

Although the way both frameworks work differs, they share some fundamentals:

It's worth adding that both frameworks can also run tests in parallel, although Swift Testing makes it easier with its struct-based approach.

Of course, there are also differences:

- XCTest runs synchronous tests on the main actor unless you say otherwise, whereas Swift Testing can run them anywhere it wants. This might mean you need to use `@MainActor` more, depending on your code.
- Swift Testing lets us selectively force specific tests to be run serially – i.e., one by one.
- Swift Testing makes it trivial to set time limits for tests. This was *possible* with XCTest if you did it by hand, so the Swift Testing approach is a big improvement.
- Swift Testing allows test suites to be actors if you want.

In the following chapters, we'll explore testing Swift concurrency using both XCTest and Swift Testing, but if you have the choice I'd suggest using Swift Testing – the extra functionality it offers for organizing tests is really invaluable.

60. How to write basic async tests

Both Swift Testing and XCTest allow us to test asynchronous code built with Swift concurrency by marking tests as `async`. This means we can wait for concurrent code to complete using `await`, we can create tasks and task groups, we can use `async let`, and more. I'll show you how it's done first in Swift Testing, then in XCTest.

Important: Regardless of whether you use XCTest or Swift Testing, it's important you use `@testable import YourAppTarget` in your test code to make all your internal types available for testing.

For Swift Testing, the main change to make is marking your `@Test` function as being `async`, like this:

```
struct DataHandling {
    @Test("Loading view model names")
    func loadNames() async throws {
        let viewModel = ViewModel()
        try await viewModel.loadNames()
        #expect(viewModel.names.isEmpty == false, "Names should be full of values.")
    }
}
```

If we were to make the same with XCTest, we'd write this:

```
final class DataHandlingTests: XCTestCase {
    func test_loadNames() async throws {
        let viewModel = ViewModel()
        try await viewModel.loadNames()
        XCTAssertFalse(viewModel.names.isEmpty, "Names should be full of values.")
    }
}
```

In both places, using `await` inside tests marks a potential suspension point, just like in production code. This potentially allows the test system to execute other tests while the first one is suspended, which in the case of Swift Testing is straightforward given that it was designed for parallel test execution as standard.

If the expectation you're making is a *requirement* – if the whole test run should be considered a failure because the current test has failed – then in Swift Testing code you should use `#require` rather than `#expect`, like this:

```
try #require(viewModel.names.isEmpty == false, "Names should be full of values.")
```

That check will throw an error on failure, which in turn will cause Swift Testing to exit the current test function.

In comparison, XCTest had a Boolean `continueAfterFailure` property on `XCTestCase`, which *sort of* did the same – as long as you were constantly toggling it each test, and didn't forget one by accident.

61. How to handle concurrency errors in unit tests

Both Swift Testing and XCTest give us three main options for handling errors in tests: we can consider them immediate failures, we can handle them internally and make our own assertions, or we can consider them expected.

First, let's look at the simplest solution in both frameworks, which is considering thrown errors to be test failures. This is done by marking your test case with `throws` and *not handling the error* – the failing test error will propagate upwards to Swift Testing and XCTest, which will fail that test.

As an example, we might be working with an error type such as this one:

```
enum LoadError: Error {
    case notEnoughData
    case tooMuchData
}
```

In Swift Testing, we could write a test to call a method that might throw one of those errors:

```
@Test("Loading view model names")
func loadNames() async throws {
    let viewModel = ViewModel()
    try await viewModel.loadNames()
    #expect(viewModel.names.isEmpty == false, "Names should be full of values.")
}
```

There is no code in there to handle the error being thrown, but that's okay: if `loadNames()` completes successfully then the expectation will be checked as normal, but if it throws an error then the test will be failed with a message such as `Caught error: .notEnoughData`.

With XCTest, the approach is identical – mark your test with `throws` and let any errors propagate upwards, like this:

```
final class DataHandlingTests: XCTestCase {
    func test_loadNames() async throws {
        let viewModel = ViewModel()
        try await viewModel.loadNames()
        XCTAssertFalse(viewModel.names.isEmpty == false, "No data was loaded.")
    }
}
```

That will perform identically to Swift Testing: either the method completes successfully and the assertion is checked, or an error is thrown and the test fails with a simple message.

While this approach is definitely appealing for its simplicity, I'm not a big fan of it because the error doesn't say exactly what was expected.

Swift Testing does offer a better solution here, though, which is to add a retroactive conformance to `CustomTestStringConvertible` in your test target. To be clear, this should be *in your test target* not in your main production code.

For example, we could clarify what our two load errors actually mean like this:

```
extension LoadError: @retroactive CustomTestStringConvertible {
    public var testDescription: String {
        switch self {
        case .notEnoughData:
            "At least three names should be loaded."
        case .tooMuchData:
            "No more than 1000 names are supported."
        }
    }
}
```

XCTest does not have an equivalent for this feature.

However, for both XCTest *and* Swift Testing an alternative is to handle the error internally and issue your own message as needed.

For Swift Testing this is done using `Issue.record()`, passing in the comment you want to flag up and also optionally also the error itself:

```
@Test("Loading view model names")
func loadNames() async {
    let viewModel = ViewModel()

    do {
        try await viewModel.loadNames()
        #expect(viewModel.names.isEmpty == false, "Names should be full of values.")
    } catch {
        Issue.record(error, "Between 3 and 1000 names are supported.")
    }
}
```

In XCTest the equivalent is catching the error manually, then calling `XCTFail()` like this:

```
final class DataHandlingTests: XCTestCase {
    func test_loadNames() async {
        let viewModel = ViewModel()

        do {
            try await viewModel.loadNames()
            XCTAssertFalse(viewModel.names.isEmpty == false, "No data was loaded.")
        } catch {
            XCTFail("Between 3 and 1000 names are supported.")
        }
    }
}
```

Tip: For both Swift Testing and XCTest, handling the errors inside the text means the test itself is no longer marked `throws`.

If the error is *expected* to be thrown – if your test is checking that an error is thrown when invalid data is provided, for example – then you'll need to take a slightly different approach.

With Swift Testing, you can run some code and expect that a specific error type is thrown:

```
@Test("Loading view model names")
func loadNames() async {
    let viewModel = ViewModel()

    await #expect(throws: LoadError.self, "At least three names should be loaded.", performing: viewModel.loadNames
}
```

That will fail the test if `LoadError` *isn't* thrown. You can even expect a specific error case to be thrown by using `LoadError.notEnoughData` rather than `LoadError.self`:

```
@Test("Loading view model names")
func loadNames() async {
    let viewModel = ViewModel()

    await #expect(throws: LoadError.notEnoughData, "At least three names should be loaded.", performing: viewModel.
}
```

In XCTest things are a little trickier: there is an `XCTAssertThrowsError()` function, but it doesn't support async functions. So, instead we should implement `do`/`catch` inside the test, then use `XCTFail()` to report a test failure if the code didn't throw:

```
final class DataHandlingTests: XCTestCase {
    func test_loadNames() async throws {
        let viewModel = ViewModel()
```

```
        do {
            try await viewModel.loadNames()
            XCTFail("This should fail to load.")
        } catch {
            // doing nothing means the test passed
        }
    }
}
```

Swift Testing provides one extra useful option, which is the `withKnownIssue()` function. This expects an error to be thrown and will in fact fail your test if the error *isn't* thrown.

Here's how it looks:

```
@Test("Loading view model names")
func loadNames() async {
    let viewModel = ViewModel()

    await withKnownIssue("Names can sometimes come back with too few values") {
        try await viewModel.loadNames()
        #expect(viewModel.names.isEmpty == false, "Names should be full of values.")
    }
}
```

This has a variation that is particularly useful while you're debugging networking code, which is the `isIntermittent` parameter – setting that to true will tell Swift to pass the test if no error is thrown, but mark an expected failure if an error *is* thrown, so it's a nice middle ground while you're tackling a problem.

We could switch our current code over to intermittent failures like this:

```
@Test("Loading view model names")
func loadNames() async {
    let viewModel = ViewModel()

    await withKnownIssue("Names can sometimes come back with too few values", isIntermittent: true) {
        try await viewModel.loadNames()
        #expect(viewModel.names.isEmpty == false, "Names should be full of values.")
    }
}
```

62. How to test completion handlers with swift testing and xctest

When you're dealing with older concurrency code that relies on callback functions that get run when some work completes rather than using Swift concurrency's `async`/`await` approach, it's important your test code waits fully for the completion handler to be called, then makes any assertions against the result of that completion handler.

As an example, we might have a class like this one:

```
class ViewModel {
    func loadReadings(completion: @escaping ([Double]) -> Void) {
        let url = URL(string: "https://hws.dev/readings.json")!

        URLSession.shared.dataTask(with: url) { data, response, error in
            if let data {
                if let numbers = try? JSONDecoder().decode([Double].self, from: data) {
                    completion(numbers)
                    return
                }
            }

            completion([])
        }.resume()
    }
}
```

That has a `loadReadings()` method that will fetch, decode, and return some data through a completion handler, although in practice your tests will likely add some kind of mock data here to avoid networking during test runs.

Testing this correctly depends on which testing framework you're using. With Swift Testing, you should use a continuation that resumes when the completion handler is called, like this:

```
@Test("Loading view model readings")
func loadReadings() async {
    let viewModel = ViewModel()

    await withCheckedContinuation { continuation in
        viewModel.loadReadings { readings in
            #expect(readings.count >= 10, "At least 10 readings must be returned.")
            continuation.resume()
        }
    }
}
```

With XCTest, you should use `XCTestExpectation` instead, like this:

```
final class DataHandlingTests: XCTestCase {
    func test_loadViewModelReadings() async {
        let viewModel = ViewModel()
        let expectation = XCTestExpectation(description: "Check view model readings")

        viewModel.loadReadings { readings in
            if readings.count >= 10 {
                expectation.fulfill()
            }
        }

        await fulfillment(of: [expectation], timeout: 10)
    }
}
```

Either way, the important things are:

63. How to test asyncsequence and asyncstream

Concurrent Swift code often streams values over time rather than returning them all at once, so it's important to be able to write tests to check an `AsyncStream`, `AsyncSequence`, or similar sends back a certain number of values. We can do this in both Swift Testing and XCTest, using *confirmations* and *expectations* respectively.

As an example, let's say we were trying to test the following `AsyncSequence`, which doubles numbers until the value wraps around, at which point it stops:

```
struct DoubleGenerator: AsyncSequence, AsyncIteratorProtocol {
    var current = 1

    mutating func next() async -> Int? {
        defer { current &*= 2 }

        if current < 0 {
            return nil
        } else {
            return current
        }
    }

    func makeAsyncIterator() -> DoubleGenerator {
        self
    }
}
```

Tip: `&*=` multiples with overflow, meaning that rather than running out of room when the value goes beyond the highest number of a 64-bit integer, it will instead flip around to be negative. We use this to our advantage, returning `nil` when we reach that point.

To test that with Swift Testing, we need to use `confirmation(expectedCount:)` to say how many doubles we expect to be sent back to us. When we use `confirmation()`, we'll be given a testing confirmation function that we call to say "yes, the work completed successfully," which in the case of our generator means it sent a value back.

The "expected count" part refers to how often we call that testing conformation function. So, here we know that our generator can double a 64-bit integer 63 times, so we could say we expect the confirmation to be called 63 times, then loop over the generator and call the confirmation each time a value comes back:

```
@Test("DoubleGenerator should create 63 doubles")
func testDoubling() async {
    let generator = DoubleGenerator()

    await confirmation(expectedCount: 63) { confirm in
        for await _ in generator {
            confirm()
        }
    }
}
```

There are three important things you need to be aware of with this code:

To get the same result with XCTest we need to use `XCTestExpectation` with two specific properties set: `expectedFulfillmentCount` to say that we expect 63 fulfillments of the test (equivalent to Swift Testing's confirmations), and `assertForOverFulfill` to say that fulfilling 64 or more times is unwanted – that we expect *exactly* 63 fulfillments.

Here's that in code:

```
final class AsyncSequenceTests: XCTestCase {
    func test_doubleGeneratorContainsCorrectValues() async {
        let generator = DoubleGenerator()
        let expectation = XCTestExpectation(description: "DoubleGenerator should create 63 doubles")
        expectation.expectedFulfillmentCount = 63
        expectation.assertForOverFulfill = true

        for await _ in generator {
            expectation.fulfill()
        }
```

```
        await fulfillment(of: [expectation], timeout: 1)
    }
}
```

Notice the very short timeout here – even 1 second is a long time to double an integer 63 times, but it's better than XCTest's default of no limit on time.

64. How to set a time limit for concurrent tests

Both Swift Testing and XCTest have simple mechanisms for controlling how long tests should be allowed to run before being considered a failure. This is critically important for concurrent tests, where the system could otherwise be waiting huge lengths of time.

With Swift Testing, time limits are set by adding an extra trait to the `@Test` macro called `.timeLimit()`. This lets you specify how many minutes – yes, *minutes* – the test should be allowed to run for before it's considered a failure.

For example, we could apply a 1-minute maximum runtime like this:

```
@Test("Loading view model names", .timeLimit(.minutes(1)))
func loadNames() async {
    let viewModel = ViewModel()
    await viewModel.loadNames()
    #expect(viewModel.names.isEmpty == false, "Names should be full of values.")
}
```

If you use a time limit with a whole test suite, that limit is applied to all tests inside there individually. If you then use a different time limit for a specific test, the shorter of the two is used.

Over in XCTest, we can do something similar with `XCTestExpectation`, but we need to wrap our async work in a task so that we have control over the execution time – if we used `await` directly then the test would stop while the work happened, regardless of the execution time limit.

So, the XCTest code would look like this:

```
final class DataHandlingTests: XCTestCase {
    func test_loadNames() async {
        let viewModel = ViewModel()
        let expectation = XCTestExpectation(description: "Names should be full of values.")

        Task {
            await viewModel.loadNames()

            if viewModel.names.isEmpty == false {
                expectation.fulfill()
            }
        }

        await fulfillment(of: [expectation], timeout: 60)
    }
}
```

The `fulfillment(of:timeout:)` function measures its timeout in seconds rather than minutes, so I've used 60 there to match the Swift Testing code.

Important: Unit tests work best when they are *fast* – you should be able to execute thousands of them a second. This means trying to perform live networking in frequently run unit tests is a bad idea, and you should either carve those tests off into a separate test target you run less frequently, or set up test mocks that let you send back virtual data immediately rather than waiting for the network.

65. How to force concurrent tests to run on a specific actor

Swift Testing and XCTest behave differently when it comes to running tests in parallel: unless you say otherwise, Swift Testing will run both synchronous and asynchronous tests on any task it likes, whereas XCTest will run asynchronous tests on background tasks will always run synchronous code on the main actor.

With Swift Testing, we have multiple options. First, we can mark individual tests with `@MainActor` or some other global actor, like this:

```
@MainActor
@Test("Loading view model names")
    func loadNames() async {
        // test code here
    }
}
```

Second, we can mark whole test suites with the same attribute, like this:

```
@MainActor
struct DataHandlingTests {
    @Test("Loading view model names")
        func loadNames() async {
            // test code here
        }
    }
}
```

And third, if you're using `confirmation()` or `withKnownIssue()` you can specify a global actor to use for just that closure, allowing the rest of the test to run elsewhere. Here's how that looks:

```
@Test("Loading view model names")
func loadNames() async {
    await withKnownIssue("Names can sometimes come back with too few values") { @MainActor in
        // test code here
    }
}
```

If you have a custom actor to work with, you can also pass it to the `isolation` parameter of both `withKnownIssue()` and `confirmation()`.

In XCTest we similar options, including the ability to mark whole tests cases as running on a specific global actor:

```
@MainActor
final class DataHandlingTests: XCTestCase {
    func test_iceCreamAllEaten() async {
        // test code here
    }
}
```

Or adding the same annotation to individual test cases:

```
final class DataHandlingTests: XCTestCase {
    @MainActor
    func test_iceCreamAllEaten() async {
        // test code here
    }
}
```

To get something similar to `confirmation()` where only one specific closure of work runs on the main actor, we'd need to create a task and isolate it there, like this:

```
final class DataHandlingTests: XCTestCase {
    func test_loadNames() async throws {
        let viewModel = ViewModel()
        let expectation = XCTestExpectation(description: "Names should be full of values.")

        Task { @MainActor in
            await viewModel.loadNames()

            if viewModel.names.isEmpty == false {
                expectation.fulfill()
            }
        }

        await fulfillment(of: [expectation], timeout: 60)
    }
}
```

66. How to serialize parameterized tests with swift testing

Swift Testing's parameterized tests allow us to send multiple values into a single test, and they'll be run in parallel for maximum performance. If you don't want this – if you want to run parameterized tests serially rather than in parallel, you should add the `.serialized` trait to the `@Test` macro.

Important: This only affects parameterized tests, and will have no effect on non-parameterized tests.

As an example, you might have a `Player` type that tracks scores, but you know it doesn't handle parallel access very well – perhaps it uses a server that limits connections, for instance.

In this case, we could use the `.serialized` test trait to perform our tests. Even though each test itself uses `async`/`await`, Swift Testing won't allow multiple instances of the test to run in parallel:

```
@Test("Scores should always be in the range 0...100", .serialized, arguments: [0, 50, 100, 200, -1]
)
func addingPoints(score: Int) async {
    var player = Player()
    await player.add(points: score)
    #expect(player.score >= 0 && player.score <= 100)
}
```

This feature is only available in Swift Testing – to get something similar in XCTest you'd need to create separate test targets and manually enable or disable parallel testing for each one.

67. How to download json from the internet and decode it into any codable type

Fetching JSON from the network and using `Codable` to convert it into native Swift objects is probably the most common task for any Swift developer, usually followed by displaying that data in a `List` or `UITableView` depending on whether they are using SwiftUI or UIKit.

Well, using Swift's concurrency features we can write a small but beautiful extension for `URLSession` that makes such work just a single line of code – you just tell iOS what data type to expect and the URL to fetch, and it will do the rest.

To add some extra flexibility, we can also provide options to customize decoding strategies for keys, data, and dates, providing sensible defaults for each one to keep our call sites clear for the most common usages.

Here's how it's done:

```
extension URLSession {
    func decode<T: Decodable>(
        _ type: T.Type = T.self,
        from url: URL,
        keyDecodingStrategy: JSONDecoder.KeyDecodingStrategy = .useDefaultKeys,
        dataDecodingStrategy: JSONDecoder.DataDecodingStrategy = .deferredToData,
        dateDecodingStrategy: JSONDecoder.DateDecodingStrategy = .deferredToDate
    ) async throws  -> T {
        let (data, _) = try await data(from: url)

        let decoder = JSONDecoder()
        decoder.keyDecodingStrategy = keyDecodingStrategy
        decoder.dataDecodingStrategy = dataDecodingStrategy
        decoder.dateDecodingStrategy = dateDecodingStrategy

        let decoded = try decoder.decode(T.self, from: data)
        return decoded
    }
}
```

That does several things:

To use the extension in your own code, first define a type you want to work with, then go ahead and call `decode()` in whichever way you need:

```
extension URLSession {
    func decode<T: Decodable>(
        _ type: T.Type = T.self,
        from url: URL,
        keyDecodingStrategy: JSONDecoder.KeyDecodingStrategy = .useDefaultKeys,
        dataDecodingStrategy: JSONDecoder.DataDecodingStrategy = .deferredToData,
        dateDecodingStrategy: JSONDecoder.DateDecodingStrategy = .deferredToDate
    ) async throws  -> T {
        let (data, _) = try await data(from: url)

        let decoder = JSONDecoder()
        decoder.keyDecodingStrategy = keyDecodingStrategy
        decoder.dataDecodingStrategy = dataDecodingStrategy
        decoder.dateDecodingStrategy = dateDecodingStrategy

        let decoded = try decoder.decode(T.self, from: data)
        return decoded
    }
}

struct User: Codable {
    let id: UUID
    let name: String
    let age: Int
}

struct Message: Codable {
    let id: Int
    let user: String
    let text: String
}

do {
    // Fetch and decode a specific type
```

```
        let url1 = URL(string: "https://hws.dev/user-24601.json")!
        let user = try await URLSession.shared.decode(User.self, from: url1)
        print("Downloaded \(user.name)")

        // Infer the type because Swift has a type annotation
        let url2 = URL(string: "https://hws.dev/inbox.json")!
        let messages: [Message] = try await URLSession.shared.decode(from: url2)
        print("Downloaded \(messages.count) messages")

        // Create a custom URLSession and decode a Double array from that
        let config = URLSessionConfiguration.default
        config.requestCachePolicy = .reloadIgnoringLocalAndRemoteCacheData
        let session = URLSession(configuration: config)

        let url3 = URL(string: "https://hws.dev/readings.json")!
        let readings = try await session.decode([Double].self, from: url3)
        print("Downloaded \(readings.count) readings")
    } catch {
        print("Download error: \(error.localizedDescription)")
    }
```

Download this as an Xcode project

As you can see, with that small extension in place it becomes trivial to fetch and decode any type of `Codable` data with just one line of Swift.